

AI 제조 기술 관련 교육

『2과목』 클린코드 + 테스트 기반 구현

2025.12.08-2025.12.19(10일 70시간)

Prepared by Daekyeong Kim

Ph.D.

Copyright 2022. Daekyeong all rights reserved

『2과목』

클린코드 + 테스트 기반 구현

- Part 1. 오리엔테이션 및 목표
- Part 2. 클린코드 핵심 원칙
- Part 3. 테스트 기반 개발
- Part 4. 종합 실습 & 마무리



학습목표

- Clean Code 원칙을 이해하고 적용한다
 - 읽기 쉬운 코드 = 유지보수 가능한 코드
- 테스트 기반 사고로 설계한다
 - 테스트는 명세이자 안전망
- PRD에서 FastAPI + Pytest 코드를 생성한다
 - Cursor AI로 자동화, 사람은 검증과 개선에 집중
- 커버리지 85% 이상 달성
 - 핵심 로직의 신뢰성 확보

순서

- Clean Code 원칙과 테스트 기반 개발(TDD) 사고를 바탕으로
- PRD에서 FastAPI + Pytest 기반 코드를 생성총 학습시간: 7시간

순번	주제
1	오리엔테이션 및 목표
2	클린코드 핵심 원칙
3	테스트 기반 개발
4	프로젝트 구조 & 모델 생성
5	API & 비즈니스 로직 구현
6	테스트 자동화 & 커버리지
7	종합 실습 & 마무리

『2과목』

클린코드 + 테스트 기반 구현

- Part 1. 오리엔테이션 및 목표
- Part 2. 클린코드 핵심 원칙
- Part 3. 테스트 기반 개발
- Part 4. 종합 실습 & 마무리





1. 클린코드란?

다음 중 클린 코드와 가장 거리가 먼 것은?

1. 적절한 행 길이를 유지하고 있는 코드
2. 한 클래스에 모든 레이어의 기능을 작성한 코드
3. 잘 작동하고 있는 테스트 코드를 가지고 있는 코드
4. 하나의 동작을 담당하도록 설계된 코드



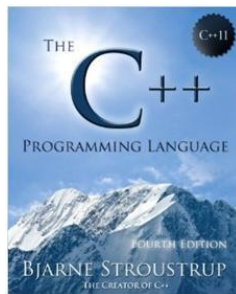
1. 클린코드란?

다음 중 클린 코드와 가장 거리가 먼 것은?

1. 적절한 행 길이를 유지하고 있는 코드
2. 한 클래스에 모든 레이어의 기능을 작성한 코드
3. 잘 작동하고 있는 테스트 코드를 가지고 있는 코드
4. 하나의 동작을 담당하도록 설계된 코드

1. 클린코드란?

- Bjarne Stroustrup, inventor of C++



“

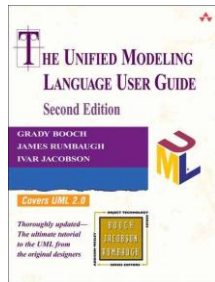
나는 우아하고 효율적인 코드를 좋아한다. 논리가 간단해야 버그가 숨어들지 못한다. 의존성을 최대한 줄여야 유지보수가 쉬워진다. 오류는 명백한 전략에 의거해 철저히 관리한다. 성능은 최적으로 유지해야 사람들이 원칙 없는 최적화로 코드를 망치려는 유혹에 빠지지 않는다.

깨끗한 코드는 한 가지를 제대로 한다.

”

1. 클린코드란?

- Grady Booch,
- author of Object-Oriented Analysis and Design with Applications



“

깨끗한 코드는 단순하고 직접적이다. 깨끗한 코드는 잘 쓴 문장처럼 읽힌다.

깨끗한 코드는 결코 설계자의 의도를 숨기지 않는다.

오히려 명쾌한 추상화와 단순한 제어문으로 가득하다.

”

1. 클린코드란?

- Dave Thomas,
- author of The Manifesto for Agile Software Development



“

깨끗한 코드는 작성자가 아닌 사람도 읽기 쉽고 고치기 쉽다.

단위 테스트 케이스와 인수 테스트 케이스가 존재한다.

깨끗한 코드에는 의미 있는 이름이 붙는다. 특정 목적을 달성하는 방법은 하나만 제공한다.

의존성은 최소이며 각 의존성을 명확히 정의한다. API는 명확하며 최소로 줄였다.

언어에 따라 필요한 모든 정보를 코드만으로 명확히 표현할 수 없기에

코드는 문학적으로 표현해야 마땅하다.

”



1. 클린코드란?

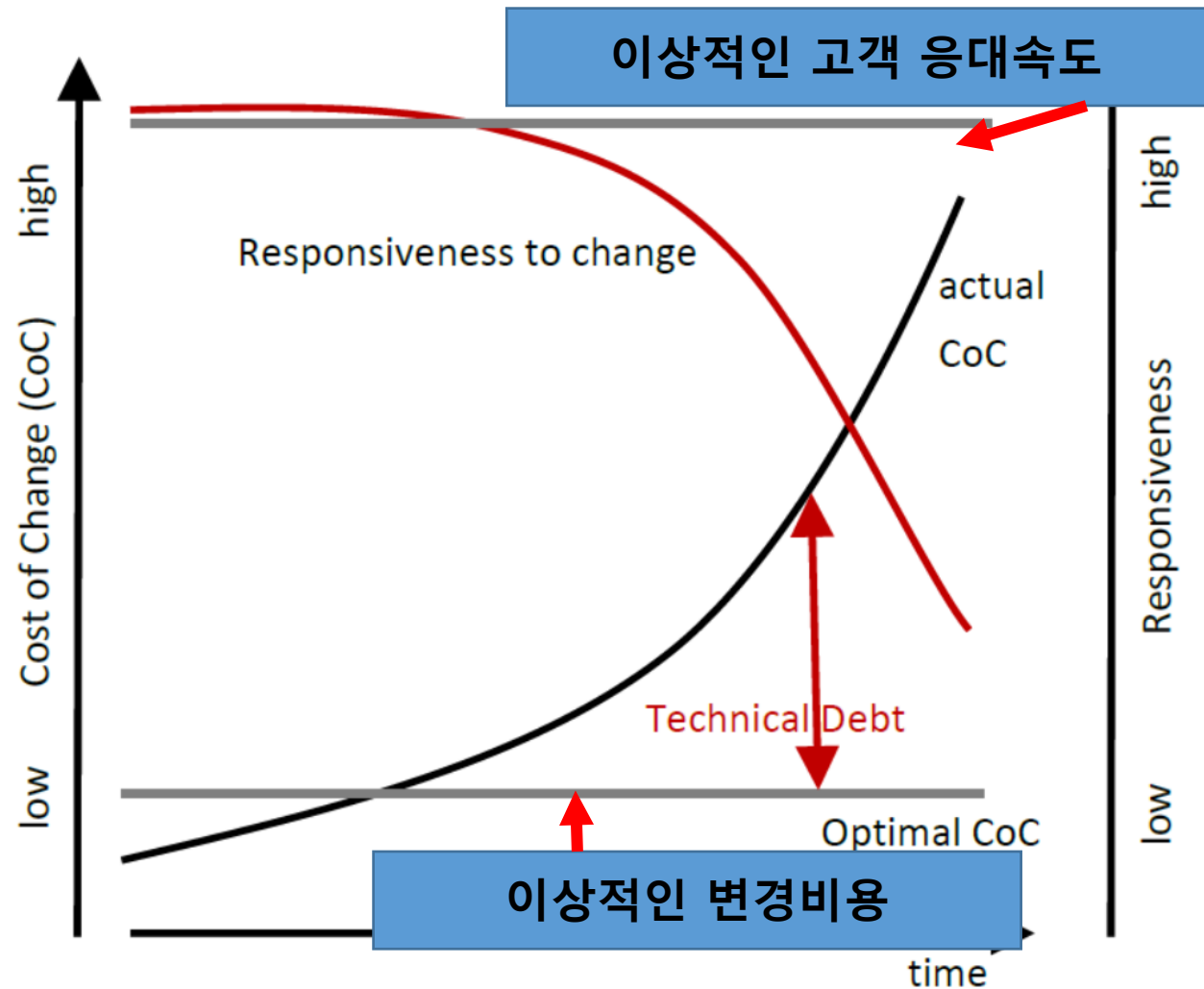
| 정의

“모든 팀원이 이해(understandability)하기 쉽도록 작성된 코드”

Readability, Changeability, Extensibility, Maintainability...

1. 클린코드란?

Clean Code를 작성해야 하는 이유





1. 클린코드란?

클린코드의 핵심

- 다른 사람이 읽기 쉬운 코드
- 버그가 숨을 곳이 없는 코드
- 변경이 두렵지 않은 코드

Follow Standard Conventions

Coding 표준, 아키텍처 표준 및 설계 가이드를 준수하라.

Keep It Simple, Stupid (KISS)

단순한 것이 효율적이며, 복잡함을 최소화하라.

The Boy Scout Rule

캠프장은 처음 왔을 때보다 더 깨끗하게 해놓고 떠나라.
(참조되거나 수정되는 코드는 원래보다 clean하게 해야 함)

Root Cause Analysis

항상 근본적인 원인을 찾아라. 그렇지 않으면 반복될 것이다

Do Not Multiple Languages in One Source File

하나의 언어는 하나의 파일로 작성
(Java, JavaScript, Html...)

2. 클린코드 원칙

① 명확한 이름

- Key Point: 변수/함수명만 봐도 의미를 알 수 있어야 한다

✗ Bad

```
def calc(o, c):  
    d = o * c.pct  
    return o - d
```

✓ Good

```
def calculate_discounted_price(original_price: float, coupon: Coupon) -> float:  
    discount_amount = original_price * coupon.discount_percentage  
    return original_price - discount_amount
```

2. 클린코드 원칙

② 중복 제거 (DRY)

Bad

주문 A 계산

```
subtotal_a = sum(item.price * item.quantity for item in order_a.items)
discount_a = subtotal_a * coupon.discount_percentage
total_a = subtotal_a - discount_a
```

주문 B 계산

```
subtotal_b = sum(item.price * item.quantity for item in order_b.items)
discount_b = subtotal_b * coupon.discount_percentage
total_b = subtotal_b - discount_b
```

Good

```
def calculate_order_total(order: Order, coupon: Optional[Coupon] = None) -> OrderTotal:
    subtotal = sum(item.price * item.quantity for item in order.items)
    discount = subtotal * coupon.discount_percentage if coupon else 0
    return OrderTotal(subtotal=subtotal, discount=discount, total=subtotal - discount)
```

2. 클린코드 원칙

③ 단일 책임 원칙 (SRP)

❌ Bad - 한 함수가 너무 많은 일을 함

```
def process_order(user_id, items, coupon_code):  
    # 1. 사용자 검증  
    user = db.query(User).filter_by(id=user_id).first()  
    if not user:  
        raise ValueError("User not found")  
  
    # 2. 쿠폰 검증  
    coupon = db.query(Coupon).filter_by(code=coupon_code).first()  
  
    # 3. 가격 계산  
    subtotal = sum(item['price'] * item['qty'] for item in items)  
    discount = subtotal * coupon.percentage if coupon else 0  
  
    # 4. 주문 저장  
    order = Order(user_id=user_id, total=subtotal - discount)  
    db.add(order)  
    db.commit()  
  
    # 5. 이메일 발송  
    send_email(user.email, order)  
  
    return order
```


2. 클린코드 원칙

③ 단일 책임 원칙 (SRP)

✓ Good - 책임 분리

```
class OrderService:
```

```
    def create_order(self, user_id: int, items: List[OrderItem], coupon_code: Optional[str]) -> Order:
        user = self._validate_user(user_id)
        coupon = self._get_coupon(coupon_code)
        order_total = self._calculate_total(items, coupon)
        order = self._save_order(user, items, order_total)
        self._notify_user(user, order)
        return order
```

```
    def _validate_user(self, user_id: int) -> User:
        # 사용자 검증만
```

```
    def _calculate_total(self, items: List[OrderItem], coupon: Optional[Coupon]) -> OrderTotal:
        # 가격 계산만
```



2. 클린코드 원칙

④ 작은 함수

함수는 한 가지 일만, 그것을 잘 해야 한다
가이드라인

- 함수 길이: 20줄 이하 권장
- 들여쓰기 깊이: 2-3단계 이하
- 파라미터: 3개 이하 권장

Good: 작고 명확한 함수들

```
def validate_order_items(items: List[OrderItem]) -> None:
```

```
    if not items:
```

```
        raise ValueError("Order must contain at least one item")
```

```
    for item in items:
```

```
        if item.quantity <= 0:
```

```
            raise ValueError(f"Invalid quantity for {item.menu_name}")
```

2. 클린코드 원칙

| ⑤ 예외 처리

✖ Bad

```
def get_user(user_id):  
    user = db.query(User).get(user_id)  
    if user is None:  
        return None # 에러를 숨김  
    return user
```

2. 클린코드 원칙

⑤ 예외 처리

 Good

```
def get_user(user_id: int) -> User:
    user = db.query(User).get(user_id)
    if user is None:
        raise UserNotFoundException(f"User {user_id} not found")
    return user
```

```
# FastAPI에서
@app.get("/users/{user_id}")
async def read_user(user_id: int):
    try:
        return get_user(user_id)
    except UserNotFoundException as e:
        raise HTTPException(status_code=404, detail=str(e))
```

2. 클린코드 원칙

| 실습 1: Clean Code 실무 예시 문제

다음 예를, 좋은 예 (Clean Code)로 개선하세요. CleanCode.py

✗ 나쁜 예 (Bad Code)

```
def calc(x, y, z):  
    if z == 1:  
        return x + y  
    elif z == 2:  
        return x - y  
    elif z == 3:  
        return x * y  
    elif z == 4:  
        return x / y  
    else:  
        return 0
```

문제점:

- 매직 넘버 (1, 2, 3, 4)
- 함수명이 모호 (calc가 무엇을 계산?)
- 하나의 함수가 너무 많은 책임
- 테스트하기 어려움



2. 클린코드 원칙

객체 지향 프로그래밍

모듈화

- 소프트웨어를 각 기능별로 분할, 설계 및 구현하는 기법
- 모듈화를 수행하면 복잡도가 감소되고, 변경과 구현이 용이하며 성능을 향상시킨다.
- 모듈간의 기능적 독립성을 보장한다.

결합도

- 모듈간의 상호 의존하는 정도, 연관관계.

응집도

- 하나의 모듈안의 요소들이 서로 관련된 정도.

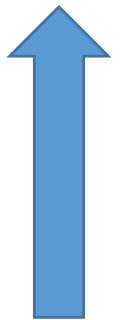
2. 클린코드 원칙

객체 지향 프로그래밍

결합도를 고려한다.

- 결합도는 모듈 간에 상호 의존하는 정도, 두 모듈 사이의 연관 관계를 말하고 결합도가 약할수록 품질이 높고, 강할수록 품질이 낮다.

결합도	내용
내용결합도	두 모듈이 코드를 공유하는 경우 내부 기능이나 그 내부 자료를 직접 참조하거나 수정하는 경우의 결합도
공통결합도	두 모듈이 전역변수를 공유하는 경우의 결합도
외부결합도	소프트웨어 외부의 환경에 좌우되는 경우의 결합도
제어결합도	한 모듈의 데이터가 다른 모듈의 실행 순서를 결정하기 위해 전달되는 경우의 결합도
스탬프결합도	배열, 레코드, 구조체 등의 자료구조가 전달될 때의 결합도.
자료결합도	모듈 간의 인터페이스 자료 요소로만 구성될때의 결합도.

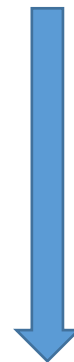


높아짐

2. 클린코드 원칙

객체 지향 프로그래밍
응집도를 고려한다.

응집도	내용
기능적 응집도	한 모듈을 구성하는 요소들이 단일 기능을 수행하기 위해 협력하는 응집도
순차적 응집도	모듈을 구성하는 한 요소의 출력이 다음 요소의 입력 데이터로 사용하는 경우의 응집도
교환적 응집도	동일한 입력과 출력을 사용하여 서로 다른 기능을 수행하는 구성요소들이 모였을 경우의 응집도
절차적 응집도	모듈이 가지는 기능들이 순서에 의해서 처리되는 경우의 응집도
시간적 응집도	특정 시간에 처리되는 기능을 모아 하나의 모듈로 작성할 경우의 응집도
논리적 응집도	모듈의 구성요소들이 유사한 기능을 수행하는 경우의 응집도
우연적 응집도	연관성이 없는 요소로만 구성되는 경우의 응집도



낮아짐

2. 클린코드 원칙

객체 지향 프로그래밍

객체 지향 프로그래밍의 주요 설계원칙

S

Single
Responsibility
Principle (SRP)

하나의 클래스는
하나의 책임만
가져야 한다.

O

Open/Closed
Principle
(OCP)

클래스는 확장에
대하여 열려
있어야 하고,
변경에 대해서
는 닫혀 있어야
한다.

L

Liskov
Substitution
Principle (LSP)

파생 클래스의
메소드는 기반
클래스의 메서드
를 대체하여 사
용될 수 있어야
한다.

I

Interface
Segregation
Principle (ISP)

클라이언트가 사
용하지 않는
메소드에
의존하지 않아야
한다.

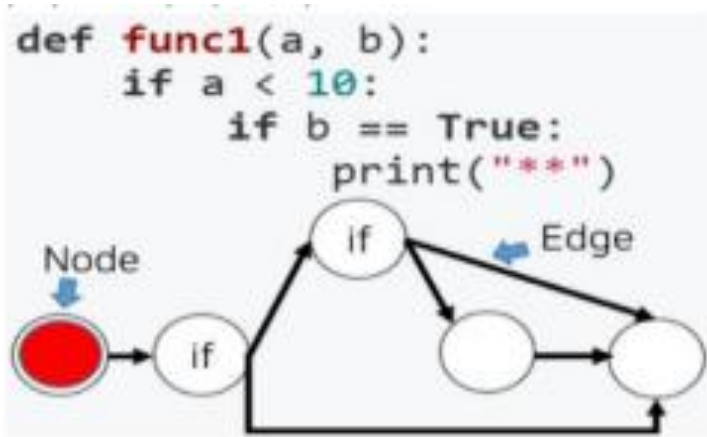
D

Dependency
Inversion
Principle (DIP)

추상화된 것은
구체적인 것에
의존하면 안된다.
(자주 변경되는
구체적인 것에
의존하지 말고
추상화된 것을
참조)

2. 클린코드 원칙

McCabe's Cyclomatic Complexity복잡도 예시



$$V(G) = E - N + 2P$$

- $V(G)$: Cyclomatic 복잡도
- E : 엣지(Edges) 수 - 흐름선
- N : 노드(Nodes) 수 - 실행 지점
- P : 독립적인 그래프의 개수 (보통 1)

- 복잡도 $V(G) = E - N + 2P$ 공식으로 E 는 그래프의 엣지, N 는 노드, P 는 연결되어 있지 않은 그래프의 부분 수로 계산한다. 따라서 위 그림의 코드 그래프는 $6(\text{Edge}) - 5(\text{Node}) + 2 = 3$ 으로 계산된다.
- 복잡도 3은 이 함수에 대해 세 가지 경로(테스트 케이스)가 필요함을 뜻한다.
- 코드가 복잡할수록 이 값은 높아지고, 이는 리팩토링 또는 함수 분리가 필요할 수 있음을 암시한다.



3. 클린코드 체크리스트

매 커밋 전 자문하기

- 변수/함수명이 명확한가?
- 중복 코드가 있는가?
- 함수가 한 가지 일만 하는가?
- 함수가 20줄을 넘는가?
- 주석 없이도 이해 가능한가?
- 예외 처리가 적절한가?
- 타입 힌트가 명시되어 있는가?



4. Clean Code의 ROI

초기 비용

작성 시간 +20%
리팩토링 노력



4. Clean Code의 ROI

장기 이익

버그 감소 -60%

유지보수 시간 -50%

온보딩 시간 -40%

코드 리뷰 시간 -30%

실무 코드 품질 체크리스트

❌ 나쁜 코드 (Code Smell)

```
def p(u, m, q): # 의미 없는 변수명
    if u and m:
        t = m.price * q
        if t > 10000: # 매직 넘버
            t = t * 0.9
        return t
    return 0
```

✅ 좋은 코드 (Clean Code)

```
def calculate_order_total(user: User, menu: Menu, quantity: int) -> Decimal:
    """주문 총액 계산 (10,000원 이상 10% 할인)"""
    DISCOUNT_THRESHOLD = Decimal("10000")
    DISCOUNT_RATE = Decimal("0.9")

    if not user or not menu:
        raise ValueError("User and Menu are required")

    subtotal = menu.price * quantity

    if subtotal >= DISCOUNT_THRESHOLD:
        return subtotal * DISCOUNT_RATE

    return subtotal
```

TDD 실무 적용 (Red-Green-Refactor)

1. RED: 실패하는 테스트 작성

```
def test_calculate_order_total_with_discount():  
    user = User(id=1, name="홍길동")  
    menu = Menu(id=1, name="아메리카노", price=4500)  
  
    # 10,000원 이상 → 10% 할인 적용  
    total = calculate_order_total(user, menu, quantity=3) # 13,500원  
  
    assert total == 12150 # 13,500 * 0.9 = 12,150  
    # ❌ 테스트 실패 (함수 미구현)
```

2. GREEN: 테스트 통과하는 최소 코드 작성

```
def calculate_order_total(user, menu, quantity):  
    subtotal = menu.price * quantity  
    if subtotal >= 10000:  
        return subtotal * 0.9  
    return subtotal  
# ✅ 테스트 통과!
```

TDD 실무 적용 (Red-Green-Refactor)

3. REFACTOR: 코드 개선

```
from decimal import Decimal
from typing import Optional
```

```
def calculate_order_total(
    user: User,
    menu: Menu,
    quantity: int,
    discount_threshold: Decimal = Decimal("10000"),
    discount_rate: Decimal = Decimal("0.9")
) -> Decimal:
    """주문 총액 계산
```

Args:

```
    user: 주문 사용자
    menu: 주문 메뉴
    quantity: 수량
    discount_threshold: 할인 기준 금액
    discount_rate: 할인율
```

Returns:

```
    Decimal: 최종 주문 금액
```

Raises:

```
    ValueError: user 또는 menu가 None인 경우
    """
```

if not user or not menu:

```
    raise ValueError("User and Menu are required")
```

```
subtotal = menu.price * quantity
```

```
if subtotal >= discount_threshold:
    return subtotal * discount_rate
```

```
return subtotal
```

 테스트 여전히 통과 + 코드 품질 향상!

| 코드 커버리지 목표

pytest-cov로 커버리지 측정

```
pytest --cov=orderbean --cov-report=html
```

목표

- 단위 테스트 커버리지: 80% 이상
- 통합 테스트 커버리지: 60% 이상
- Critical Path: 100%

커버리지 미달 시 CI 실패

```
pytest --cov=orderbean --cov-fail-under=80
```

『2과목』

클린코드 + 테스트 기반 구현

- Part 1. 오리엔테이션 및 목표
- Part 2. 클린코드 핵심 원칙
- Part 3. 테스트 기반 개발
- Part 4. 종합 실습 & 마무리





1. 왜 테스트를 작성하는가?

| 일반적 오해

- ✗ "버그를 찾기 위해"
- ✗ "QA가 시키니까"
- ✗ "커버리지를 채우기 위해"



1. 왜 테스트를 작성하는가?

진짜 이유

- ✓ 설계를 검증하기 위해
- ✓ 리팩토링의 안전망
- ✓ 살아있는 문서
- ✓ 빠른 피드백



2. 테스트란?

| 정의

S/W Testing 은 S/W 의 품질을 조사하는 과정

**Software testing is an investigation conducted to
Provide stakeholders with information about the Quality
of the product or service under test.**

**출처 : Exploratory Testing, Cem Kaner, Florida Institute of Technology, Quality Assurance
Institute Worldwide Annual Software Testing Conference, Orlando, FL, Nov 2006**

2. 테스트란?

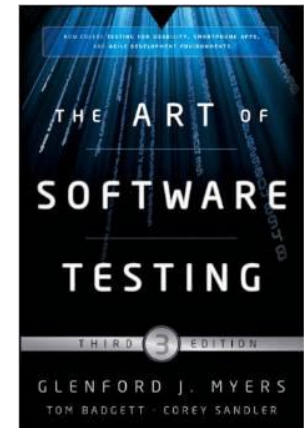
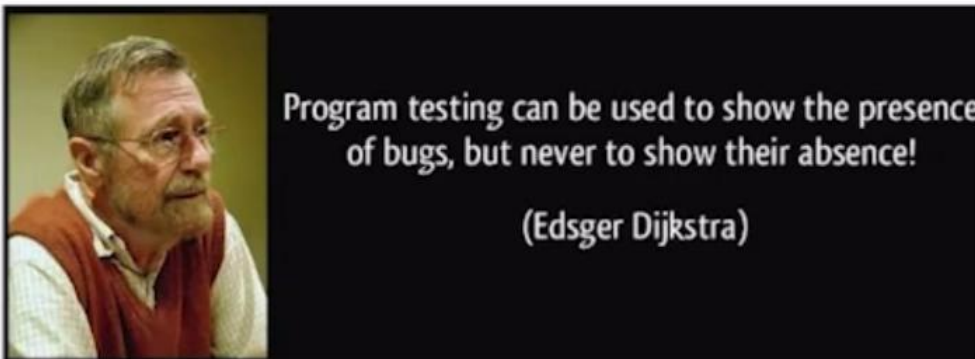
| 정의

메이어스(Myers)

- Test is the process of executing SW with the intent of finding defects.
- 테스트는 결함을 찾기 위해 SW를 실행하는 과정입니다.

다익스트라(Dijkstra)

- Test can be used to show the presence of bugs, but never to show their absence.
- 테스트는 버그의 존재 여부를 보여주는 데 사용할 수 있지만, 버그가 없다는 것을 보여주는 데는 사용할 수 없습니다.



Testing 의 종류

Functional Testing

- Unit Testing
- Integration Testing
- System Testing
- Smoke Testing
- Sanity Testing
- Interface Testing
- Regression Testing
- Beta/Acceptance Testing

Non-Functional Testing

- Performance Testing
- Load Testing
- Stress Testing
- Volume Testing
- Security Testing
- Compatibility Testing
- Install Testing
- Recovery Testing
- Reliability Testing
- Usability Testing
- Compliance Testing
- Localization Testing

2. 테스트란?

| Testing 방법

변경 후 기도하기(Edit and Pray)

- 신중한 검토 -> 변경 대상 이해 -> 변경 -> 정상 동작 확인-> 의도하지 않은 변경 여부 확인(Side effect)
- 전문적인 방법처럼 보이지만, 신중 ≠ 안정

체계적인 보호 없이 변경을 시도하는
방식으로, 위험성이 높음

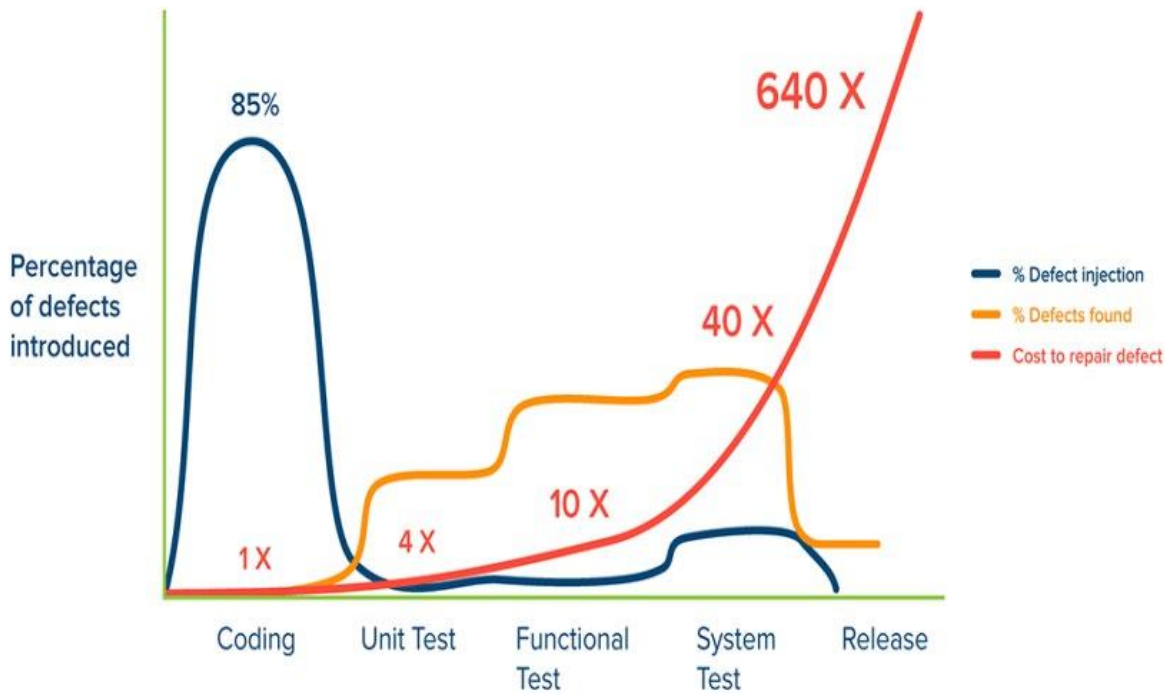
보호 후 수정하기(Cover and Modify)

- 테스트 루틴으로 코드를 보호
- 문제 발생시 빠르게 찾을 수 있음
- 테스트 루틴 = 피드백

코드 변경 후 문제를 사전에 감지하여,
Side Effect를 방지하고 안전하게 유
지보수하는 방식

2. 테스트란?

Cost of Defect



Jones, Capers. *Applied Software Measurement: Global Analysis of Productivity and Quality*.

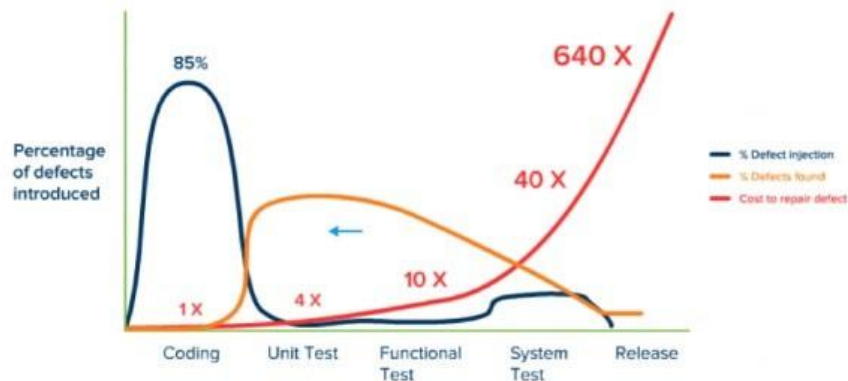
- 결함 투입 시점
: 코딩 단계
- 결함 발견 시점
: Test 단계
- 결함 수정 비용
: 개발 후반부로 갈수록 급격히 증가

2. 테스트란?

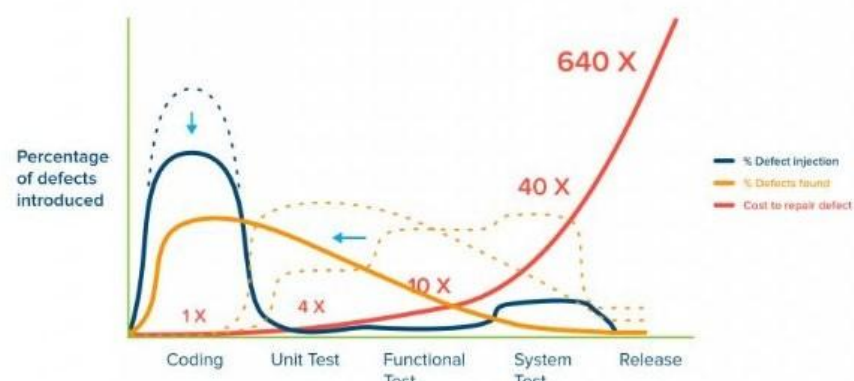
Cost of Defect

테스트를 개발 초기에 시행하여 defect 를 찾아낼수록 defect 비용은 감소한다.

- "Shift-Left Testing"은 테스트를 개발 초기(코딩 및 단위 테스트 단계)로 이동시켜서, 결함을 더 일찍 발견하고 수정 비용을 줄임
- 소프트웨어의 품질을 높이고 출시 후 문제를 최소화



Jones, Capers. Applied Software Measurement: Global Analysis of Productivity and Quality.



Jones, Capers. Applied Software Measurement: Global Analysis of Productivity and Quality.



3. 좋은 테스트의 특징

| 테스트 구조

AAA(Arrange-Act-Assert)
Given/When/Then

3. 좋은 테스트의 특징

| Given/When/Then 이슈- BDD

- 비개발자가 Cursor.ai를 활용하여 개발을 진행할 때, BDD(Behavior-Driven Development)가 가장 적합
비개발자가 Cursor.ai를 활용하여 개발을 진행할 때, BDD(Behavior-Driven Development)가 가장 적합
- 비개발자가 Cursor.ai를 활용하여 개발을 진행할 때, BDD(Behavior-Driven Development)로 **Given/When/Then** 을 쓴다.
- 사용자 행위 중심의 개발 구조를 제공한다는 점입니다. BDD는 사용자의 행동을 자연어로 시나리오화하는 접근법으로, 일반적인 "Given-When-Then" 형태의 서술을 이용합니다. 예를 들어, "사용자가 로그인된 상태 (Given)에서 + 버튼을 클릭하면(When) 할 일 목록에 새로운 항목이 추가된다(Then)"라는 시나리오를 작성하면, 이를 Cursor.ai의 프롬프트로 입력하여 곧바로 코드 구현에 활용할 수 있습니다.



3. 좋은 테스트의 특징

Given/When/Then 이슈- BDD

- TDD(Test-Driven Development)에 비해 비개발자가 접근하기 쉽다는 이점이 있습니다. TDD에서는 단위 테스트 코드 작성 자체가 어느 정도 프로그래밍 지식을 요구합니다. 하지만 BDD는 자연어 기반의 시나리오 작성이 핵심이므로 비교적 접근 장벽이 낮습니다. 무엇보다 Cursor.ai의 프롬프트 입력 방식이 BDD 시나리오 문서화 과정과 유사하기 때문에, 비개발자가 요구사항을 직접 입력하고 실시간으로 결과물을 확인하기가 수월합니다.
- 결국 BDD와 Cursor.ai를 함께 사용하면, 비개발자가 직접 기획 단계에서 작성한 요구사항을 코드로 구현해낼 수 있는 길이 열립니다. 이는 개발 비용과 시간을 획기적으로 단축하면서도 실제 사용자 입장에서 필요한 기능을 빠르게 MVP(최소 기능 제품) 수준으로 구현하는 데 큰 강점을 지닙니다. 특히 소규모 프로젝트나 스타트업 환경처럼 빠른 프로토타이핑이 필요한 경우, BDD와 Cursor.ai의 결합은 이상적인 솔루션이 될 수 있습니다.

3. 좋은 테스트의 특징

| 테스트 구조

Given/When/Then > 테스트가 스토리처럼 읽힌다

```
def test_order_with_discount_coupon():
```

```
    # Given (준비): 테스트 전제 조건
```

```
    user = create_test_user()
```

```
    menu_items = [
```

```
        {"menu_id": 1, "quantity": 2, "price": 5000}
```

```
    ]
```

```
    coupon = create_coupon(discount_percentage=0.1) # 10% 할인
```

```
    # When (실행): 테스트할 동작
```

```
    order = create_order(user.id, menu_items, coupon.code)
```

```
    # Then (검증): 예상 결과 확인
```

```
    assert order.subtotal == 10000
```

```
    assert order.discount == 1000
```

```
    assert order.total == 9000
```



3. 좋은 테스트의 특징

F.I.R.S.T

Fast: 빠르게 실행 (ms 단위)

Isolated or Independent: 독립적 실행 가능

Repeatable: 언제 어디서나 동일한 결과

Self-validating: 자동으로 성공/실패 판단

Timely: 적시에 작성 (코드와 함께)

3. 좋은 테스트의 특징

| 테스트의 6가지 영역: Right-BICEP

무엇을 테스트해야 하는가? 경계 조건은 맞는가?

- R (Right) : 결과가 옳은가?
- B (Boundary conditions) : 모든 경계조건이 정확한가?
- I (Inverse relationships) : 역관계를 확인할 수 있는가?
- C (Cross-checks) : 다른 수단을 사용해서 결과를 교차 확인할 수 있는가?
- E (Error conditions) : 에러조건을 강제로 만들어 낼 수 있는가?
- P (Performance characteristics) : 성능특성이 한도 내에 있는가?

결과가 옳은가?

- 결과의 유효성 검사
- 이 코드가 옳게 동작한다면, 어떻게 그것을 알 수 있는가?
- 데이터 파일 이용하기
- 예) 파일에서 테스트 내용을 읽어온다.

3. 좋은 테스트의 특징

CORRECT 경계 조건 검증

CORRECT?

버그는 '경계조건' 근처, 즉 그 코드가 평소의 루틴과 다르게 동작하는 조건에서 많이 발생한다.

- 형식 일치(Conformation) - 값의 형식이 예상한 형식과 일치하는가?
- 순서(Ordering) - 적절히 순서대로 되어 있거나 그렇지 않은 값인가?
- 범위(Range) - 적당한 최소 값과 최대 값 사이에 있는 값인가?
- 참조(Reference) - 코드가 자기가 직접 제어하지 않는 외부 코드를 참조하는가?
- 존재성(Existence) - 값이 존재하는가?
- 개체 수(Cardinality) - 확실히 충분한 값이 존재하는가?
- 시간(Time) - 모든 것이 순서대로 일어나는가? 제시간에? 때맞추어?



4. 테스트 케이스 설계 기법

테스트 목적을 바탕으로 ...

프로그램의 코드를 직접 실행하여 개발자 입장에서 테스트를 수행하거나 명세서를 기반으로 사용자 입장의 테스트를 수행하는 테스트 방법 등의 다양한 방법론이 존재한다.

방법론	방법론 설명	테스트 방법론 사례
동적 테스트	프로그램 실행을 요구하는 테스트	화이트박스, 블랙박스
정적 테스트	프로그램 실행 없이 구조를 분석하여 논리성 검증	코드 검사, 워크스루
화이트박스 테스트	프로그램 내부 로직을 보면서 테스트 (구조 테스트)	구조 테스트, 루프 테스트
블랙박스 테스트	프로그램 외부 명세를 보면서 테스트 (기능 테스트)	동등분할, 경계값분석, 원인결과 그래프/오류예측
하드웨어 테스트	하드웨어의 성능과 신뢰성 테스트	부하테스트, 스트레스 테스트
소프트웨어 테스트	이해관계자들에게 시험 대상 제품 또는 서비스의 품질에 관한 정보를 제공하는 조사 과정	인간은 코드의 모든 결함을 찾는 것이 불가능하다.



4. 테스트 케이스 설계 기법

블랙박스 테스트(명세 기반 테스트) 주요 방법

구문(syntax)

- 입력 데이터가 사전에 정의해 놓은 데이터 유형에 부합하는지(valid), 부합하지 않는지(invalid)를 분류한 뒤 예상되는 결과를 테스트하는 방법
- 블랙박스 테스트 방법 중에서 가장 간단한 테스트 방법

동등 분할(equivalence partitioning)

- 입력 데이터를 중심으로 테스트 케이스를 만들어 테스트하는 방법
- 입력 조건에 부합하는 데이터와 부합하지 않는 데이터의 개수를 균등하게 설정하여 테스트 케이스를 작성

페어와이즈(Pairwise) 테스트

경계값 분석(boundary value analysis)

- 입력 조건의 중앙 부근의 값보다는 경계 지점의 값에서 오류 발생의 확률이 높은 점을 감안하여 입력 조건의 경계값을 테스트 케이스에 반영하여 테스트하는 방법
- 범위 바깥의 값과 경계값으로 테스트를 수행
- 동치 분할 방법이 입력 데이터에 초점을 맞추므로 인해 분석에 일부단점이 있는 것을 보완하기 위한 방법



4. 테스트 케이스 설계 기법

블랙박스 테스트(명세 기반 테스트) 주요 방법

원인-효과 그래프(cause-effect graphing)

- 입력 데이터 간의 관계와 입력을 원인으로 출력을 효과로 보아 입력 이출력에 영향을 미치는 상황을 체계적으로 분석함으로써 효과가 높은 테스트 케이스를 선정하여 테스트하는 방법
- 동등 분할과 경계값 분석 방법이 입력 환경의 복잡성을 충분히 반영하지 못하는 문제점을 보완한 테스트 방법

비교(comparison)

- 프로그램의 형상 통제를 통해 축적한 여러 버전의 프로그램에 동일한 검사 데이터를 통해 점검하여 동일한 결과를 얻을 수 있는지 비교하는 식으로 테스트하는 방법

오류 예측(error guessing)

- 과거에 축적한 경험 데이터나 테스트 수행자의 전문성을 보완하여 테스트에 반영하는 방법
- 테스트 방법이 누락하기 쉬운 오류를 경험과 감각으로 찾아내는 방법



4. 테스트 케이스 설계 기법

블랙박스 테스트(명세 기반 테스트) 주요 방법

의사 결정 테이블(decision table)

- 입력 데이터 및 출력 데이터의 값이 참(true)과 거짓(false)으로 결정되는 경우 모든 경우의 수를 확인하는 방법
- 입력 데이터값이 적은 수의 조건일 경우에 유효

상태 천이도(state-transition diagram)

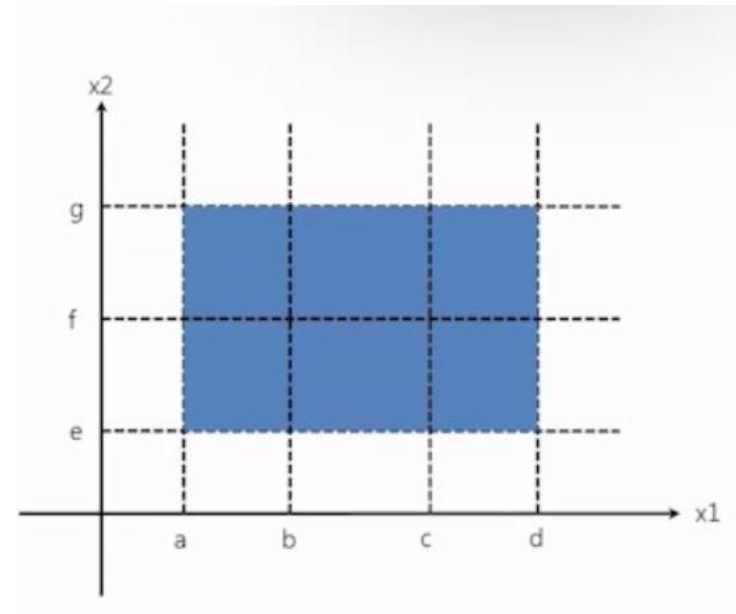
- 테스트 상황에 따라 유효한 상태 천이와 유효하지 않은 상태 천이를 수행하도록 테스트 케이스를 설계하는 방법



4. 설계 기법-블랙박스 테스트(명세 기반 테스트) 주요 방법

동등 분할(equivalence partitioning)

- 기본 아이디어
 - 입력 데이터를 동일한 동작이 예상되는 클래스로 분할한다.
 - 각 클래스의 대표값을 선택하여 테스트 데이터를 설계한다.
 - 단, 분할된 클래스에 속한 값은 모두 같다고 가정한다.
- 테스트 데이터
 - 입력값을 여러 개의 동등 클래스로 분할한다.
 - 각 클래스마다 대표값을 추출한다.
- 이점
 - 누락없이 완전하게 테스트할 수 있다.
 - 중복없이 테스트할 수 있다.





4. 설계 기법-블랙박스 테스트(명세 기반 테스트) 주요 방법

동등 분할(equivalence partitioning)

- Example 1 : 성적 판정
 - 동등 분할을 이용하여 성적(0~100) 등급을 판정하는 테스트 데이터를 작성하시오.

동등 클래스	등급
90 ~ 100	A
80 ~ 89	B
70 ~ 79	C
0 ~ 69	F

입력 값	기대 등급
95	A
85	B
75	C
35	F



4. 설계 기법-블랙박스 테스트(명세 기반 테스트) 주요 방법

동등 분할(equivalence partitioning)

- Example 2 : 무게 판정
 - 동등 분할을 이용하여 택배 무게 측정에 대한 테스트 데이터를 작성하시오.

동등 클래스	등급
$0 < \text{무게} < 3$	가
$3 \leq \text{무게} < 7$	나
$7 \leq \text{무게} < 15$	다
$15 \leq \text{무게} < 25$	라
$25 \leq \text{무게} < 30$	마

입력 값	기대 등급
1.5	가
6.3	나
8,9	다
21.7	라
27.2	마



4. 설계 기법-블랙박스 테스트(명세 기반 테스트) 주요 방법

구간



X	2	4	6
$f(x)$	1	1	1
$P(x)$	$\frac{1}{3}$	$\frac{1}{3}$	$\frac{1}{3}$

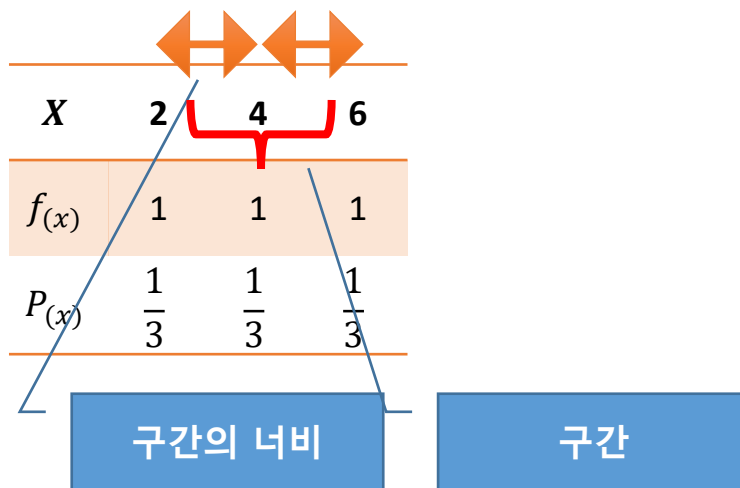
구간의 너비 구간

- 일반적으로, 각 구간의 너비는 다음과 같이 한다.
 - (최대값 - 최소값) / 구간의 개수
 - 첫 번째 구간에는 양적 자료의 최소값이 포함되도록 하고, 제일 마지막 구간에는 최대값이 포함되도록 구간을 만들면 된다. 이렇게 만들어진 구간을 이용하여 각 구간에 있는 양적 자료의 빈도와 백분율을 구해서 표를 작성하면 된다. 작성된 표를 이용하여 양적 자료에 있는 특징을 파악하면 된다.
- 그러나,



4. 설계 기법-블랙박스 테스트(명세 기반 테스트) 주요 방법

구간



- 구간을 만들 때에 구간을 몇 개로 할 지에 대해서는 다양한 방법이 있습니다. 여기서는 Herbert A. Sturges(1926)가 「The Choice of a Class Interval」라는 제목으로 발표한 논문에서 제시한 공식은 다음과 같다. 참고로 n 은 데이터의 개수를 의미한다.

$$1 + 3.3\log_{10}(n)$$

- 예 : 100개의 자료일 경우 $1 + 3.3\log_{10}(100) = 7.3 \rightarrow 7 \sim 8$ 개



4. 설계 기법-블랙박스 테스트(명세 기반 테스트) 주요 방법

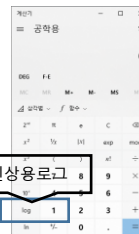
구간과 빈도

- 아래의 예제 데이터를 가지고 구간을 만들고 빈도와 백분율을 구해보도록 한다.

54 56 57 59 73 74 76 78 78 82 82 85 86 91 94 98 116

- Sturges 공식을 따르면, 구간의 개수는 다음과 같다.

$$1 + 3.3 \log_{10}(17) = 5.06$$



밑수가 10인 상용로그

밑수가 n인 자연로그

- 구간의 개수는 반올림해서 5개로 한다. 구간의 너비는 $(116-54)/5$ 로 하면 대략적으로 15가 된다.

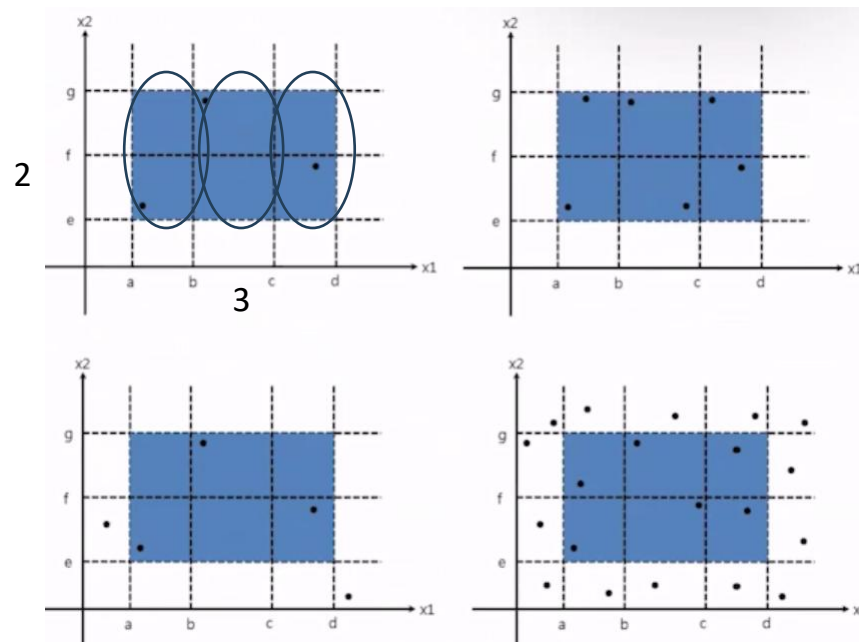
구분	빈도	백분율(%)
50이상 ~ 65이하	4	23.5
65초과 ~ 80이하	5	29.4
80초과 ~ 95이하	6	35.3
95초과 ~ 110이하	1	5.9
110초과~125이하	1	5.9
합계	17	100.0



4. 설계 기법-블랙박스 테스트(명세 기반 테스트) 주요 방법

동등 분할(equivalencepartitioning)의 다른 예

- Weak normal
 - Single fault assumption 가정\한 변수의 동치 클래스 구간만이용
 - 가장 많은 변수 선택
- Strong normal
 - Multiple fault assumption 가정
 - 모든 변수간의 곱 집합을 이용
- Weak robust
 - Weak normal + 범위 밖
- Strong robust
 - Strong normal + 범위 밖





4. 설계 기법-블랙박스 테스트(명세 기반 테스트) 주요 방법

동등 분할(equivalence partitioning)

- Example 3 : 우편물 발송
 - 동치 클래스가 주어졌을 때 Weak normal 동등 분할로 테스트 데이터를 작성하시오.

무게
$0 < \text{무게} < 3$
$3 \leq \text{무게} < 7$
$7 \leq \text{무게} < 15$
$15 \leq \text{무게} < 25$
$25 \leq \text{무게} < 30$

5

지역
A
B
C

3

유형
일반
익일

2

가장 많은 변수 선택

무게	지역	유형
1.3	A	일반
5.7	B	익일
7.2	C	일반
23.8	A	일반
26	B	익일



4. 설계 기법-블랙박스 테스트(명세 기반 테스트) 주요 방법

동등 분할(equivalence partitioning)

- Example 4 : 날짜 계산
 - Weak normal, Strong normal 동등 분할로 테스트 데이터를 작성하시오.

입력 값	결과
$1 \leq \text{month} \leq 12$ $1 \leq \text{day} \leq 31$ $1900 \leq \text{year} \leq 2100$	내일 날짜

동치 클래스

- M1 = {month : month has 30 days}
- 3 M2 = {month : month has 31 days}
- M3 = {month : month is February}
- D1 = {1 <= day <= 28}
- 4 D2 = {day = 29}
- D3 = {day = 30}
- D4 = {day = 31}
- Y1 = {year = 2000}
- 3 Y2 = {year : year is a non-century leap year}
- Y3 = {year : year is a common year}

Weak Normal

month	day	year	Expected Output
6	14	2000	6/15/2000
7	29	1996	7/30/1996
2	30	2002	Invalid Input Date
6	31	2000	Invalid Input Date

가장 많은 변수 선택



4. 설계 기법-블랙박스 테스트(명세 기반 테스트) 주요 방법

동등 분할(equivalence partitioning)

- Example 4 : 날짜 계산
- Weak normal, Strong normal 동등 분할로 테스트 데이터를 작성하시오.

Strong Normal

month	day	year	Expected Output
6	14	2000	6/15/2000
6	14	1996	6/15/1996
6	14	2002	6/15/2002
6	29	2000	6/30/2000
6	29	1996	6/30/1996
6	29	2002	6/30/2002
6	30	2000	Invalid Input Date
6	30	1996	Invalid Input Date
6	30	2002	Invalid Input Date
6	31	2000	Invalid Input Date
6	31	1996	Invalid Input Date
6	31	2002	Invalid Input Date
7	14	2000	7/15/2000
7	14	1996	7/15/1996
7	14	2002	7/15/2002
7	29	2000	7/30/2000
7	29	1996	7/30/1996
7	29	2002	7/30/2002
7	30	2000	7/31/2000
7	30	1996	7/31/1996
7	30	2002	7/31/2002

month	day	year	Expected Output
7	31	2000	8/1/2000
7	31	1996	8/1/1996
7	31	2002	8/1/2002
2	14	2000	2/15/2000
2	14	1996	2/15/1996
2	14	2002	2/15/2002
2	29	2000	Invalid Input Date
2	29	1996	3/1/1996
2	29	2002	Invalid Input Date
2	30	2000	Invalid Input Date
2	30	1996	Invalid Input Date
2	30	2002	Invalid Input Date
2	31	2000	Invalid Input Date
2	31	1996	Invalid Input Date
2	31	2002	Invalid Input Date

3x4x3

모든 변수간의
곱 집합



4. 설계 기법-블랙박스 테스트(명세 기반 테스트) 주요 방법

페어와이즈(Pairwise) 테스트

- 기본 아이디어
 - Strong normal, All combinations 경우처럼 모든 가능한 입력 값들의 조합들을 테스트하는 대신 모든 짝(pair)들의 조합을 테스트하는 방법

전화	설정	음소거
음성	일반	Off
영상	스피커	On



전화	설정	음소거
음성	일반	Off
음성	일반	On
음성	스피커	Off
음성	스피커	On
영상	일반	Off
영상	일반	On
영상	스피커	Off
영상	스피커	On



전화	설정	음소거
음성	일반	Off
음성	스피커	On
영상	일반	On
영상	스피커	Off



4. 설계 기법-블랙박스 테스트(명세 기반 테스트) 주요 방법

페어와이즈(Pairwise) 테스트

- Example : 피자주문
 - 도우:곡물,나폴리,썬 크러스트
 - 토핑:기본,프리미엄
 - 주문:전화,온라인

테스트 케이스	입력			기대 출력
	도우	토핑	주문	
1	곡물	기본	전화	13000
2	곡물	기본	온라인	12000
3	곡물	프리미엄	전화	15000
4	곡물	프리미엄	온라인	14000
5	나폴리	기본	전화	15000
6	나폴리	기본	온라인	14000
7	나폴리	프리미엄	기본	17000
8	나폴리	프리미엄	온라인	16000
9	썬 크러스트	기본	전화	17000
10	썬 크러스트	기본	온라인	16000
11	썬 크러스트	프리미엄	전화	19000
12	썬 크러스트	프리미엄	온라인	18000

입력 인자 값

입력 인자	도우	토핑	주문
동등 클래스	곡물	기본	전화
	나폴리	프리미엄	온라인
	썬 크러스트		

테스트 케이스	입력			기대 출력
	도우	토핑	주문	
1	곡물	기본	전화	13000
2	곡물	프리미엄	온라인	12000
3	나폴리	기본	전화	15000
4	나폴리	프리미엄	온라인	14000
5	썬 크러스트	기본	온라인	18000
6	썬 크러스트	프리미엄	전화	19000

Pairwise 테스트케이스

All combinations 테스트케이스



4. 설계 기법-블랙박스 테스트(명세 기반 테스트) 주요 방법

경계값 분석(Boundary Value Analysis)

- 기본 아이디어
 - 에러는 경계 값 근처에서 빈번히 발생한다.
 - 그러므로, 경계 값 주위를 꼼꼼히 조사하자.
 - 각 클래스의 하한 및 상한에서 데이터를 도출한다.
- 테스트 데이터
 - 1-value 경계값: min, max
 - 2-value 경계값: min-, min, max, max+
 - 하한 경계값과 하한 경계를 벗어난 가장 가까운 값
 - 상한 경계값과 상한 경계를 벗어난 가장 가까운 값



4. 설계 기법-블랙박스 테스트(명세 기반 테스트) 주요 방법

경계값 분석(Boundary Value Analysis)

- Example 1 : 성적 판정
 - 1-value, 2-value 경계값 분석으로 성적 등급에 대한 테스트 데이터를 작성하시오.

명세

점수	등급
90 ~ 100	A
80 ~ 89	B
70 ~ 79	C
0 ~ 69	F

1-value 경계값 분석

하한경계	상한경계	기대 등급
90	100	A
80	89	B
70	79	C
0	69	F

2-value 경계값 분석

하한경계	상한경계	기대 등급
89, 90	100, 101	
79, 80	89, 90	
69, 70	79, 80	
-1, 0	69, 70	



4. 설계 기법-블랙박스 테스트(명세 기반 테스트) 주요 방법

경계값 분석(Boundary Value Analysis)

- Example 2 : 무게 판정
 - 2-value 경계값 분석으로 택배 무게 측정에 대한 테스트 데이터를 작성하시오.

명세

무게	등급
$0 < \text{무게} < 3$	가
$3 \leq \text{무게} < 7$	나
$7 \leq \text{무게} < 15$	다
$15 \leq \text{무게} < 25$	라
$25 \leq \text{무게} < 30$	마

2-value 경계값 분석

무게	등급	하한 2-Value 경계값	상한 2-Value 경계값
$0 < \text{무게} < 3$	가	0, 0.1	2.9, 3
$3 \leq \text{무게} < 7$	나	2.9, 3	6.9, 7
$7 \leq \text{무게} < 15$	다	6.9, 7	14.9, 15
$15 \leq \text{무게} < 25$	라	14.9, 15	24.9, 25
$25 \leq \text{무게} < 30$	마	24.9, 25	29.9, 30



4. 설계 기법-블랙박스 테스트(명세 기반 테스트) 주요 방법

경계값 분석(Boundary Value Analysis)

- Example 3 : 소수 판정
 - 0~1000 값을 받아 들어 소수를 판정하는 프로그램의 테스트 데이터를 작성하시오.

입력 값	기대 결과
0	No
1000	No

1-value 경계값

입력 값	기대 결과
-1	Invalid
0	No
1000	No
1001	Invalid

2-value 경계
값

번호	입력 값	기대 결과
1	-1	음수이어서 도움 메시지 출력
2	0	No
4	3	Yes
5	1000	No
6	1001	범위 넘어선 값이어서 도움 메시지 출력
7	2 7	두 개 이상 입력이어서 도움 메시지 출력
8	"r"	문자이어서 도움 메시지 출력
9	공백	공백이어서 도움 메시지 출력



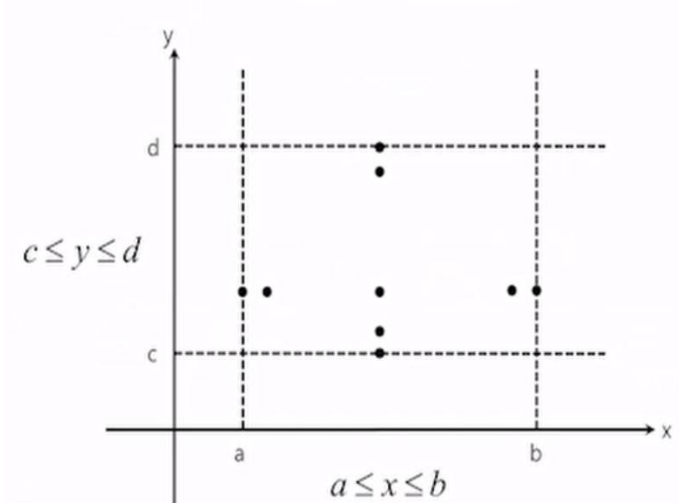
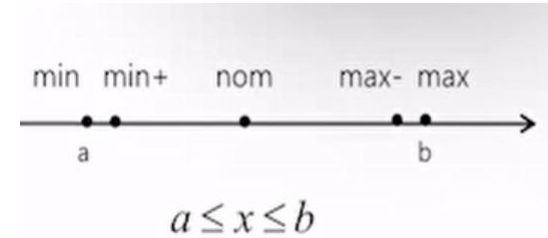
4. 설계 기법-블랙박스 테스트(명세 기반 테스트) 주요 방법

기타 테스트 데이터

- 중앙값(nom)을 테스트해 볼 수도 있음
- 3-value인 min-, min, min+, max-, max, max+ 를 취할 수 있음

기타 사용되는 경계값 분석

- Basic BVA(Boundary Value Analysis)
- Robust BVA
- Worst-case BVA
- Robust Worst-case BVA

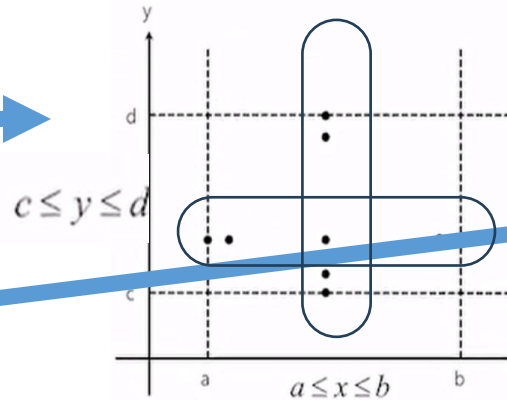




4. 설계 기법-블랙박스 테스트(명세 기반 테스트) 주요 방법

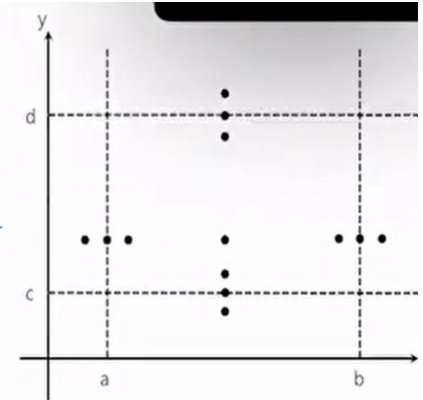
Basic BVA(Boundary Value Analysis)

- 기본적인 경계값 고려
- Single fault 채택



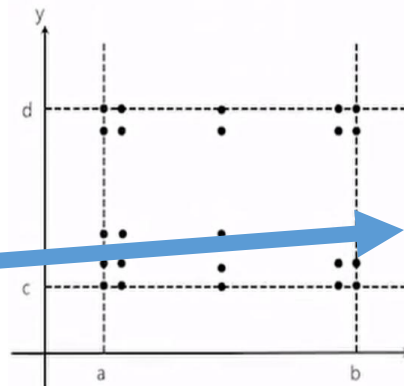
Robust BVA

- Basic BVA 확장
- 범위 밖 고려



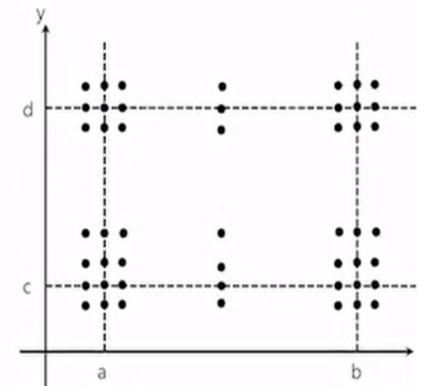
Worst-case BVA

- 모든 조합을 고려
- Multiple fault 허용



Robust Worst-case BVA

- Robust + Worst-case





4. 설계 기법-블랙박스 테스트(명세 기반 테스트) 주요 방법

기타 사용되는 경계값 분석

- Example 4 : 삼각형 판정
 - Basic BVA로 테스트 데이터를 작성하시오.

입력 값	결과
$0 \leq a \leq 200$	정삼각형
$0 \leq b \leq 200$	이등변삼각형
$0 \leq c \leq 200$	부등변삼각형
	삼각형 아님(Invalid)

테스트 시

a	b	c	기대 결과
100	100	1	이등변삼각형
100	100	2	이등변삼각형
100	100	100	정삼각형
100	100	199	이등변삼각형
100	100	200	삼각형 아님
100	1	100	이등변삼각형
100	2	100	이등변삼각형
100	100	100	정삼각형
100	199	100	이등변삼각형
100	200	100	삼각형 아님
1	100	100	이등변삼각형
2	100	100	이등변삼각형
100	100	100	정삼각형
199	100	100	이등변삼각형
200	100	100	삼각형 아님



4. 설계 기법-블랙박스 테스트(명세 기반 테스트) 주요 방법

의사 결정 테이블 혹은 결정표(decision table)

- 기본 아이디어
 - 입력 조건의 모든 조합을 고려한다.
 - 입력 조건 조합에 따른 출력을 결정한다.
- 테스트 데이터
 - 입력 조건을 모두 나열한다.
 - 조건에 따른 출력을 결정한다.
- 이점
 - 복잡한 논리적 관계를 표현하기 좋음
 - 누락된 요구사항 검사에 유용

테스트 조건		1	2	3	4
입력값	유효한 아이디	T	T	F	F
	유효한 비밀번호	T	F	T	F
예상 출력값	로그인 성공	T	F	F	F
	잘못된 아이디 경고창	F	F	T	T
	잘못된 비밀번호 경고창	F	T	F	F

테스트 케이스	입력값 조합	예상 동작
1번	둘 다 맞음 (T/T)	로그인 성공
2번	아이디 O / 비번 X	비밀번호 경고창
3번	아이디 X / 비번 O	아이디 경고창
4번	둘 다 틀림 (F/F)	아이디 경고창 (우선 순위가 아이디)



4. 설계 기법-블랙박스 테스트(명세 기반 테스트) 주요 방법

의사 결정 테이블 혹은 결정표(decision table)

- Example 1 : 급여 계산
 - 근무 시간과 시급
 - $0 < \text{hour} < 80$
 - $0 < \text{wage} < 100$
 - 초과 수당 지급
 - $\text{hour} > 40$ 이면 1.5배 지급
 - 출력
 - 정상급여, 초과수당
 - 시간예러
 - 시급예러

테스트 조건	1	2	3	4	5	6
$0 < \text{hour}$	T	T	T	F	T	T
$\text{hour} < 80$	T	T	F	T	T	T
$40 < \text{hour}$	F	T	T	F	x	x
$0 < \text{wage}$	T	T	T	T	T	F
$\text{wage} < 100$	T	T	T	T	F	T
출력	정상급여	초과수당	시간예러	시간예러	시급예러	시급예러

hour	wage	expected pay
30	40.00	1200.00
60	50.00	3500.00 (overtime)
90	60.00	invalid hour
50	100.00	invalid wage
...	...	...



4. 설계 기법-블랙박스 테스트(명세 기반 테스트) 주요 방법

의사 결정 테이블 혹은 결정표(decision table)

• Example 2 : 온라인 주문

— 결재

- 현금 또는 신용카드(둘 다 결재하지 않는다)

— 우수고객

- 10% 할인

— 출력

- 주문 접수
- 주문 거부
- 10% 할인
- 정상 가격

		조건의 조합							
		현금 결제	신용카드 결제	우수 고객	주문 접수	주문 거부	10% 할인	정상 가격	
조건	현금 결제	Y	Y	Y	Y	N	N	N	N
	신용카드 결제	Y	Y	N	N	Y	Y	N	N
	우수 고객	Y	N	Y	N	Y	N	Y	N
출력	주문 접수			O	O	O	O		
	주문 거부							O	O
	10% 할인			O		O			
	정상 가격				O		O		

3 4 5 6 7

		조건의 조합					
		현금 결제	신용카드 결제	우수 고객	주문 접수	주문 거부	10% 할인
조건	현금 결제	Y	Y	N	N	N	
	신용카드 결제	N	N	Y	Y	N	
	우수 고객	Y	N	Y	N	-	
출력	주문 접수	O	O	O	O		
	주문 거부					O	
	10% 할인	O		O			
	정상 가격		O		O		

3 4 5 6 7

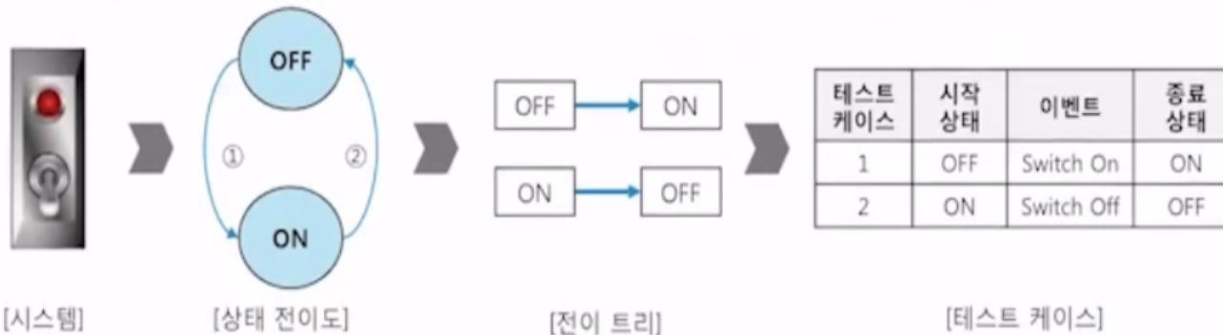
입력			예상 출력
현금결제	신용카드	우수고객	
Y	N	Y	10% 할인 현금 주문 접수
Y	N	N	정상 가격 현금 주문 접수
N	Y	Y	10% 할인 신용카드 주문 접수
N	Y	N	정상 가격 신용카드 주문 접수
N	N	-	주문 거부



4. 설계 기법-블랙박스 테스트(명세 기반 테스트) 주요 방법

상태 전이도(state-transition diagram)

- 기본 아이디어
 - 시스템의 상태 전이도를 기반으로 테스트 데이터를 도출
- 테스트 데이터
 - 대상 시스템의 상태 전이도를 작성함
 - 상태 전이도를 이용해 전이 트리를 만듦
 - 전이 트리로부터 각 전이를 테스트하는 테스트 케이스를 작성함



• 상태 커버리지

$$= \frac{\text{테스트 한 상태 수}}{\text{전체 상태 수}} \times 100\%$$
$$= \frac{2}{2} \times 100\% = 100\%$$

• 전이 커버리지

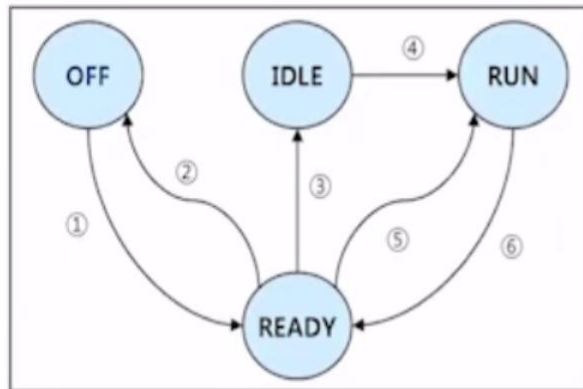
$$= \frac{\text{테스트 한 전이 수}}{\text{전체 전이 수}} \times 100\%$$
$$= \frac{2}{2} \times 100\% = 100\%$$



4. 설계 기법-블랙박스 테스트(명세 기반 테스트) 주요 방법

상태 전이도(state-transition diagram)

- Example : 프로세스 제어기
 - 상태 커버리지, 전이 커버리지 100%를 만족시키는 테스트 케이스를 구하시오.



[상태 전이도]

$$\frac{\text{테스트한 상태 수}}{\text{전체 상태 수}} \times 100\% = \frac{4}{4} \times 100\% = 100\%$$

$$\frac{\text{테스트한 전이수}}{\text{전체 상태 수}} \times 100\% = \frac{6}{6} \times 100\% = 100\%$$



4. 설계 기법-블랙박스 테스트(명세 기반 테스트) 주요 방법

시나리오 테스트

- 기본 아이디어
 - 실 사용하는 방식에 기반하여 흐름을 테스트함
 - 시스템이 실 사용되는 프로세스 흐름에서 결함을 발견하는 데 유용함
 - End-to-End testing
 - 시스템 테스트, 인수 테스트 단계에서 주로 사용함
- 기본(main, basic) 시나리오
 - 가장 일반적이고 정상적인 상황을 나타내는 흐름
- 대체(alternative)시나리오
 - 기본 시나리오 외 시나리오
 - 사용자 혹은 시스템에 의해 선택적으로 수행하는 흐름
 - 비정상적인 사용, 시스템에서 발생하는 에러 등을 처리하는 흐름



4. 설계 기법-블랙박스 테스트(명세 기반 테스트) 주요 방법

시나리오 테스트

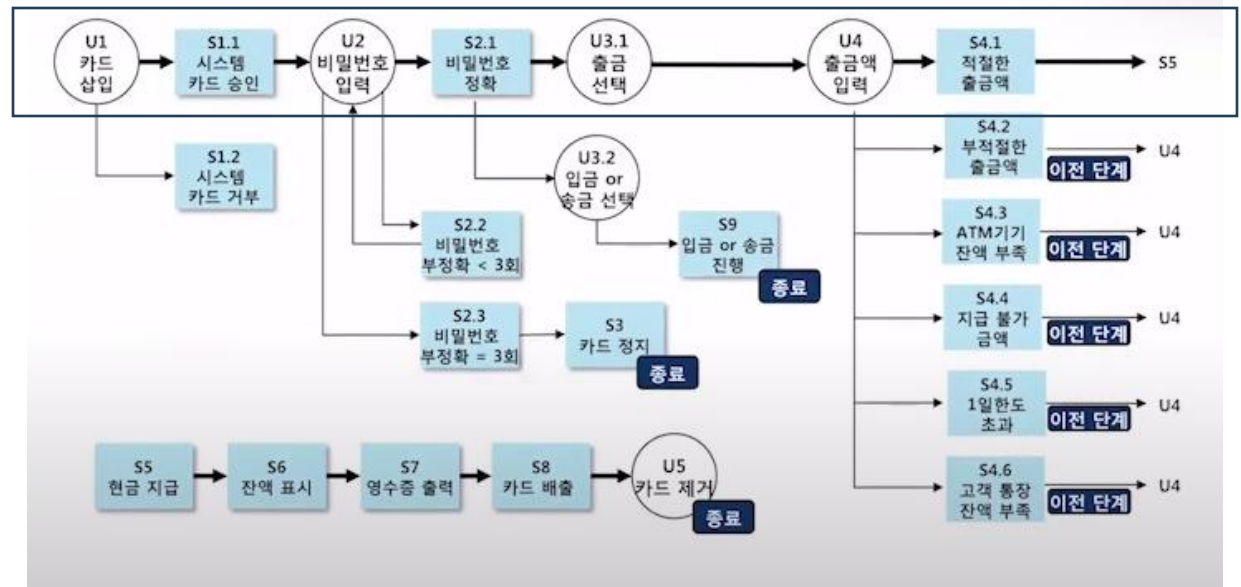
- Example1 : ATM 현금 인출 시나리오
 - 기본 시나리오
 - 계좌에서 성공적으로 현금 인출
 - 대체 시나리오
 - 유효하지 않은 카드
 - 비밀번호 입력 오류 2회
 - 비밀번호 입력 오류 3회, 거래 정지
 - 출금 대신 입금이나 계좌이체를 선택
 - 잘못된 출금 액 선택(예: 만원 단위로 출금이 가능하나, 만오천원 출금)
 - ATM 기기의 잔액이 충분하지 않음
 - 지급 권종 부족(예: 3만원 인출하고자 하나, 5만원권만 있는 경우)
 - 1일 한도 초과
 - 고객 계좌 잔액 부족



4. 설계 기법-블랙박스 테스트(명세 기반 테스트) 주요 방법

시나리오 테스트

- Example1 : ATM 현금 인출 시나리오
- ATM 현금 상태 전이 다이어그램





4. 설계 기법-블랙박스 테스트(명세 기반 테스트) 주요 방법

시나리오 테스트

- Example1 : ATM 현금 인출 시나리오
- 기본 시나리오 테스트 케이스

항목	내용
테스트 케이스#	1
테스트 케이스 이름	현금인출 성공
시나리오 실행 경로	U1, S1.1, U2, S2.1, U3.1, U4, S4.1, S5, S6, S7, S8, U5
입력값	- 유효한 카드 - 293910982246 이 올바르다고 가정 - 유효한 비밀번호 - 1234 가 올바르다고 가정 - ATM 잔액 - \$50,000 - 고객 계좌 잔액 - \$100 - 출금액 - \$50
사전 조건	- 유효한 카드이고 올바른 비밀번호이며, ATM 기기가 정상동작 중일 때만 출금이 가능함 - 고객 계좌로부터 성공적으로 인출됨
기대 결과	- ATM 잔액 - \$49,950 - 고객 계좌 잔액 - \$50 - ATM 운영 중, 고객 카드 입력 대기 중



4. 설계 기법-블랙박스 테스트(명세 기반 테스트) 주요 방법

시나리오 테스트

- Example1 : ATM 현금 인출 시나리오
 - 대체 시나리오 테스트 케이스

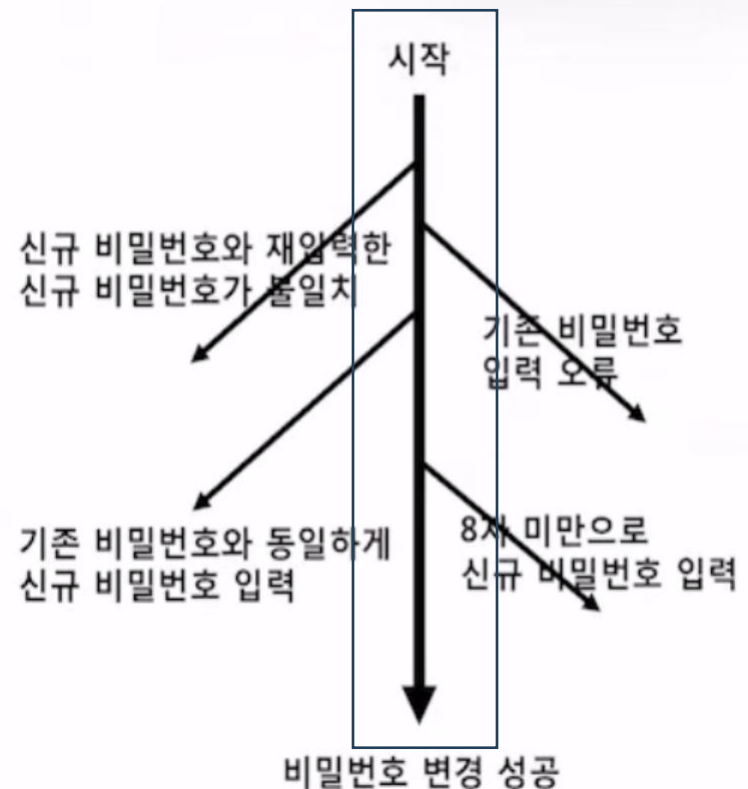
항목	내용
테스트 케이스#	2
테스트 케이스 이름	유효하지 않은 카드
시나리오 실행 경로	U1, S1.2, S8, U5
입력값	유효하지 않은 카드
사전 조건	유효한 카드이고 올바른 비밀번호이며, ATM 기기가 정상동작 중일 때만 출금이 가능함
기대 결과	- 카드는 ATM 기기에서 거부되고 카드가 유효하지 않다는 에러 메시지가 표시됨 - 고객이 유효하지 않은 카드를 제거하도록 ATM 기기는 대기 중상태로 전환됨



4. 설계 기법-블랙박스 테스트(명세 기반 테스트) 주요 방법

시나리오 테스트

- Example2 : 비밀번호 변경 시나리오
 - 기본 시나리오
 - 비밀번호를 성공적으로 변경함(8자리 이상)
 - 대체 시나리오
 - 기존 비밀번호 입력 오류 발생함
 - 8자 미만으로 신규 비밀번호를 입력함
 - 기존 비밀번호와 동일하게 신규 비밀번호를 입력함
 - 신규 비밀번호와 재입력한 신규 비밀번호가 불일치함





4. 설계 기법-화이트박스(구조 기반 테스트) 테스트

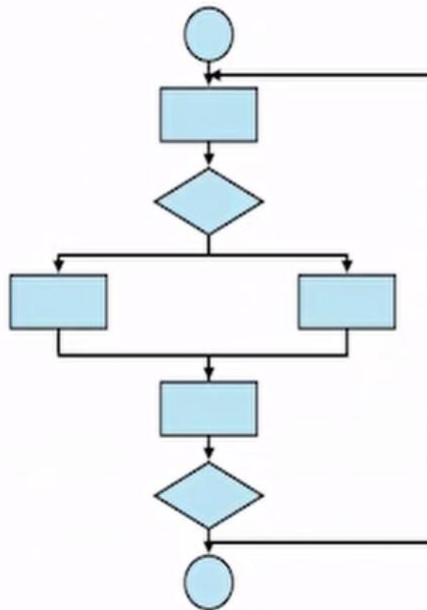
- 기초 경로(basic path)
 - 순환 복잡도(cyclomatic complexity)를 이용하여 테스트하는 방법 · 테스트 대상 경로의 입출력 경로를 선택함으로써 테스트 커버리지를 달성하는 형태로 접근
- 제어 흐름
 - 프로그램의 제어 구조를 그래프 형태로 파악하고 그래프의 구성 요소로 문장(statement), 제어 블록(control block), 분기(branch), 조건(condition), 다중 조건(multiple condition) 등을 인식하여 테스트하는 방법
- 데이터 흐름(data flow)
 - 제어 흐름 그래프에 데이터의 사용 현황을 추가한 그래프 형태로 파악하여 테스트하는 방법
- **테스트 커버리지(Test Coverage)**란 소프트웨어의 코드, 기능, 요구 사항 등이 테스트를 통해 얼마나 검증되었는지를 측정하는 지표입니다. 테스트가 얼마나 철저하게 수행되었는지를 나타내는 지표로, 버그를 줄이고 소프트웨어의 품질을 높이는 데 중요한 역할을 합니다.



4. 설계 기법-화이트박스(구조 기반 테스트) 테스트

- 1) 문장 커버리지
- 2) 결정(분기) 커버리지
- 3) 조건 커버리지
- 4) 결정/조건 커버리지
- 5) 다중 조건 커버리지
- 6) MC/DC
- 7) 모든 경로 커버리지

- 기본 경로 테스트



구조 기반 테스트

= 화이트 박스 테스트

= 코드 기반 테스트

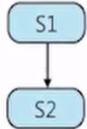
≠ 블랙 박스 테스트



4. 설계 기법-화이트박스(구조 기반 테스트) 테스트

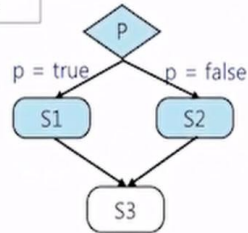
• 제어 흐름도(CFG, Control Flow Graph)

S1
S2



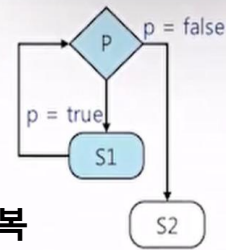
순차

if p S1 else S2



분기

while (p) S1

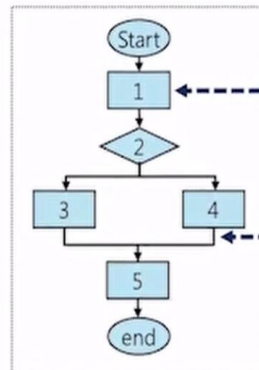


반복

[프로그램 코드]

```
INT X = 1;  
IF ( X > 0 ) THEN  
    X = X - 1;  
ELSE  
    X = X + 1;  
ENDIF  
PRINT(X);
```

[제어 흐름도]



노드(Node)

* 노드: 프로그램 실행, 분기,
제어 흐름 시작과 끝

엣지(Edge)

* 엣지: 노드 간 흐름



4. 설계 기법-화이트박스(구조 기반 테스트) 테스트

문장 커버리지

모든 코드의 문장(statement)이 최소 한 번 이상 실행되었는지를 측정하는 테스트 커버리지 지표

- Example1 : 다음 코드의 100% 문장 커버리지를 만족하는 테스트 데이터는?

입력값 : X, Y, Z

```
IF (( X > 1) AND ( Y == 1))
```

```
    Z = Z / X
```

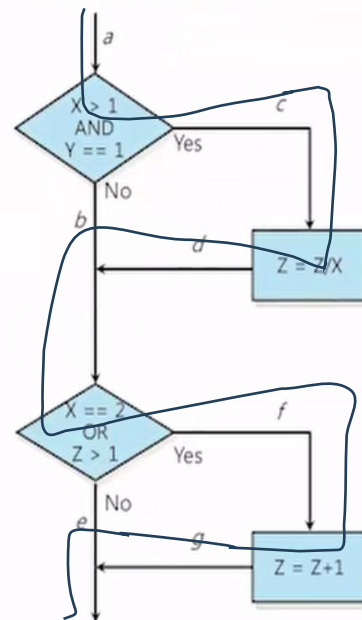
```
END
```

```
IF ((X == 2) OR ( Z > 1))
```

```
    Z = Z + 1
```

```
END
```

1. 제어 흐름도 작성



2. 모든 노드를 지나는
경로를 식별하고 **최소개**의
테스트 데이터 집합 식별

~~<테스트 데이터>
X=3, Y=0, Z=1
X=2, Y=1, Z=6~~



<테스트 데이터>
X=2, Y=1, Z=6

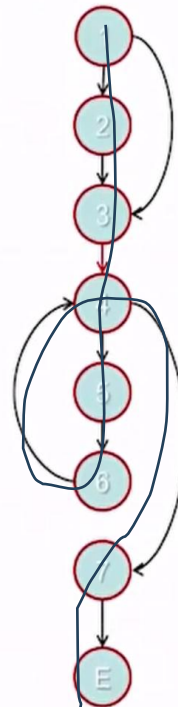


4. 설계 기법-화이트박스(구조 기반 테스트) 테스트

문장 커버리지

- Example2 :
 - 다음 코드의 100% 문장 커버리지를 만족하는 테스트 데이터는?

```
if (x < 0) /* 1 */ {  
    x := -x /* 2 */  
};  
z := 1; /* 3 */  
while (x > 0) /* 4 */ {  
    z := z * y; /* 5 */  
    x := x - 1 /* 6 */  
};  
return z /* 7 */
```



한 경로이면 충분한다

[1,2,3,4,5,6,4,7,E]

이 경로를 지나게 할 최소 개 입력값은?

x = -1
y = 어떤 값도 괜찮음
z = 어떤 값도 괜찮음

(x=-1, y=2, z=3)



4. 설계 기법-화이트박스(구조 기반 테스트) 테스트

결정 커버리지

if, while, for 등 분기 조건이 true/false 두 방향 모두 수행되었는지 확인

- Example1 : 다음 코드의 100% 결정 커버리지를 만족하는 테스트 데이터는?

입력값 : X, Y, Z

```
IF (( X > 1) AND ( Y == 1))
```

```
    Z = Z / X
```

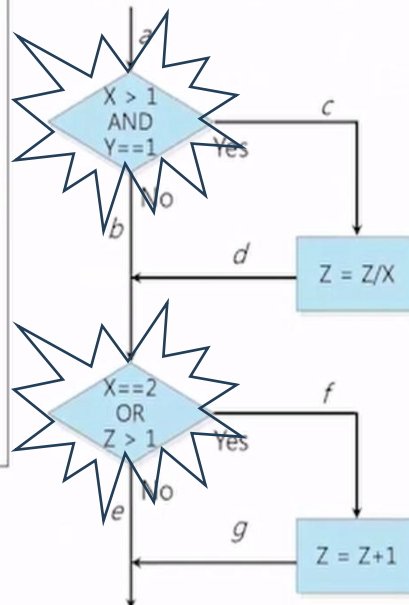
```
END
```

```
IF ((X == 2) OR ( Z > 1))
```

```
    Z = Z + 1
```

```
END
```

1. 제어 흐름도 작성



2. 모든 분기를 지나는
경로를 가지는 **최소개**의
테스트 데이터 집합 식별

<테스트 데이터>

X=2, Y=1, Z=6

X=3, Y=0, Z=1

1: YES, YES로 가는 경로

2: No, No로 가는 경로

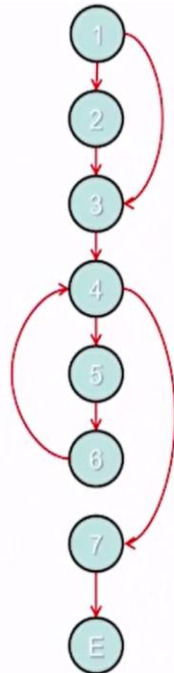


4. 설계 기법-화이트박스(구조 기반 테스트) 테스트

결정 커버리지

- Example2 :
 - 다음 코드의 100% 결정커버리지를 만족하는 테스트 데이터는?

```
if (x < 0) /* 1 */ {  
  x := -x /* 2 */  
};  
z := 1; /* 3 */  
while (x > 0) /* 4 */ {  
  z := z * y; /* 5 */  
  x := x - 1 /* 6 */  
};  
return z /* 7 */
```



최소 두 경로가 있어야 한다.

p1 = [1,2,3,4,5,6,4,7,E]
p2 = [1,3,4,7,E]

이들 두 경로를 지나게 할 최 개 입력값은?

p1 = (x=-1, y=*, z=*)
p2 = (x=0, y=*, z=*)

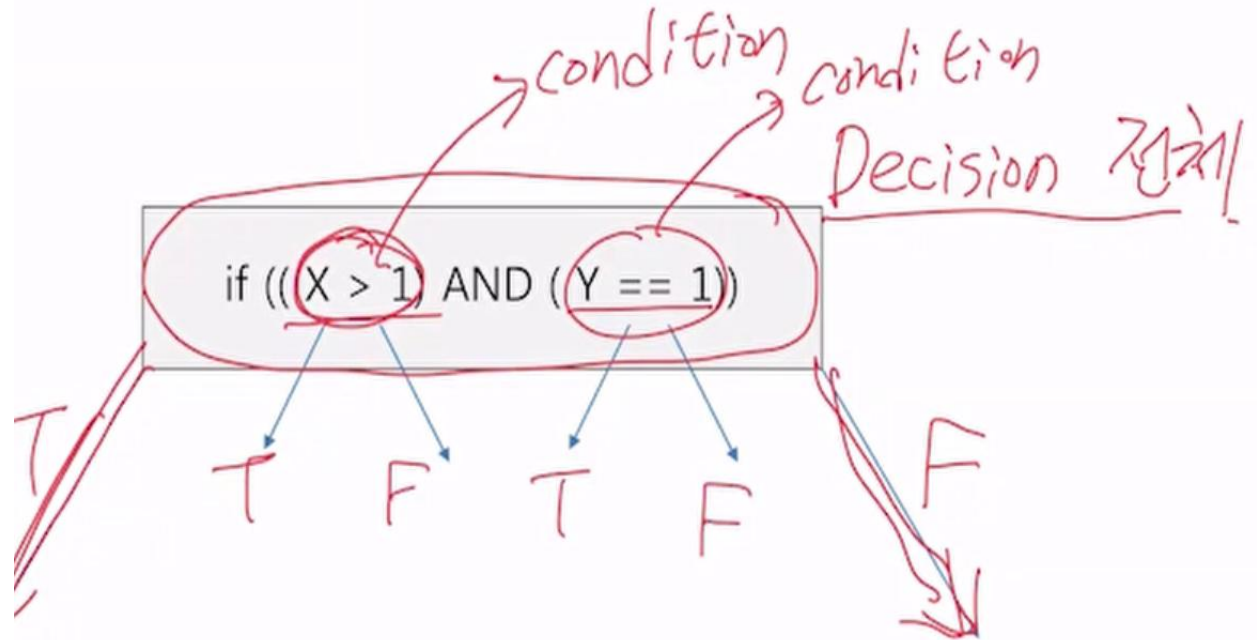
이들 데이터로 두 번 실행하면 결정커버리지 만족한다.



4. 설계 기법-화이트박스(구조 기반 테스트) 테스트

결정과 조건

- 결정(Decision)
- 조건(Condition)





4. 설계 기법-화이트박스(구조 기반 테스트) 테스트

조건 커버리지

각 개별 조건(항)이 true, false가 한 번 이상 되는지 확인

- Example : 다음 코드의 100% 조건 커버리지를 만족하는 테스트 데이터는?

입력값 : X, Y, Z

```
IF (( X > 1) AND ( Y == 1))  
    Z = Z / X  
END
```

```
IF ((X == 2) OR ( Z > 1))  
    Z = Z + 1  
END
```

1. 개별 조건 식별

2. 모든 개별 조건의 T, F를 만족하는 최소 테스트 데이터 집합 식별

테스트 데이터	기본조건			
	X>1	Y==1	X==2	Z>1
X=2, Y=1, Z=6	true	true	true	true
X=0, Y=0, Z=1	false	false	false	false

만족

<테스트 데이터 집합>

X=2, Y=1, Z=6

X=0, Y=0, Z=1



4. 설계 기법-화이트박스(구조 기반 테스트) 테스트

결정/조건 커버리지

- Example :
- 다음 코드의 100% 조건 커버리지를 만족하는 테스트 데이터는?

입력값 : X , Y , Z

```
IF (( X > 1) AND ( Y == 1))  
    Z = Z / X  
END
```

```
IF ((X == 2) OR ( Z > 1))  
    Z = Z + 1  
END
```

1. 개별 조건과 결정문의 분기를 식별
2. 전체 조건에 유의하여 모든 개별 조건의 T, F를 만족 하는 최소개의 테스트 데이터 집합 식별

테스트 데이터	(X>1) AND (Y==1)			(X==2) OR (Z>1)		
	기본조건		전체 조건	기본조건		전체 조건
	X>1	Y==1		X==2	Z>1	
X=2, Y=1, Z=6	true	true	true	true	true	true
x=0, y=0, z=1	false	false	false	false	false	false

만족

<테스트 데이터 집합>

X=2, Y=1, Z=6

X=0, Y=0, Z=1



4. 설계 기법-화이트박스(구조 기반 테스트) 테스트

다중 조건 커버리지(MCC:Multiple Condition Coverage)

- Example :
 - 다음 코드의 100% 조건 커버리지를 만족하는 테스트 데이터는?

if (X>0 AND Y>0 AND Z>0)

TC	테스트 데이터			X>0	Y>0	Z>0
1	X=1	Y=1	Z=1	T	T	T
2	X=1	Y=1	Z=0	T	T	F
3	X=1	Y=0	Z=1	T	F	T
4	X=1	Y=0	Z=0	T	F	F
5	X=0	Y=1	Z=1	F	T	T
6	X=0	Y=1	Z=0	F	T	F
7	X=0	Y=0	Z=1	F	F	T
8	X=0	Y=0	Z=0	F	F	F

n	Multiple Conditions	Test time
1	2	2 ms
4	16	16 ms
8	256	256 ms
16	65,536	65.5 s
32	4,294,967,296	49.5 days

조건에 따라 테스트 할 수 있는 케이스 2^n 과 같다.



4. 설계 기법-화이트박스(구조 기반 테스트) 테스트

MC/DC (Modified Condition/Decision Coverage) 커버리지

- 정의
 - Modified Condition/Decision Coverage
 - 변형된 조건/결정 커버리지
 - 조건의 수 + 1개의 테스트 데이터 요구
 - 실무에서 사용되는 최고 수준의 테스트
- 활용
 - 자동차 분야 ISO 26262 ASIL D 등급 권고
 - 항공 분야 DO-178C 표준 Level A 등급 요구 사항

n	MCC	MC/DC
1	2	2
4	16	5
8	256	9
16	65,536	17
32	4,294,967,296	33

지수 2^n

전형
 $n+1$

조건에 따라 테스트 할 수 있는 케이스 $n+1$ 과 같다



4. 설계 기법-화이트박스(구조 기반 테스트) 테스트

MC/DC 커버리지

- Example 1:
 - AND 연산자 : 조건 2
 - T,F가 나오는 경우
 - $N + 1 = 3$

고정

	IF ((X > 1) AND (Y == 1))		
TC	(X>1)	(Y==1)	(X>1) AND (Y==1)
TC1	T	T	T
TC2	T	F	F
TC3	F	T	F
TC4	F	F	F

고정

TC	(X>1)	(Y==1)	(X>1) AND (Y==1)
TC1	T	T	T
TC3	F	T	F

TC	(X>1)	(Y==1)	(X>1) AND (Y==1)
TC1	T	T	T
TC2	T	F	F

TC	(X>1)	(Y==1)	(X>1) AND (Y==1)
TC1	T	T	T
TC2	T	F	F
TC3	F	T	F



4. 설계 기법-화이트박스(구조 기반 테스트) 테스트

MC/DC 커버리지

- Example 2:
 - OR 연산자 : 조건 2
 - T,F가 나오는 경우
 - $N + 1 = 3$

IF ((X == 2) OR (Z > 1))			
TC	(X==2)	(Z>1)	(X==1) OR (Z>1)
TC1	T	T	T
TC2	T	F	T
TC3	F	T	T
TC4	F	F	F



4. 설계 기법-화이트박스(구조 기반 테스트) 테스트

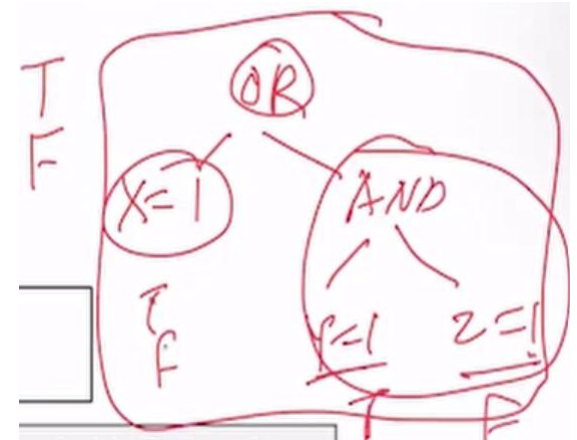
MC/DC 커버리지

- Example 3:
 - 다음 코드의 100% MC/DC 커버리지를 만족하는 테스트 데이터는?

if (X==1 OR (Y==1 AND Z==1))

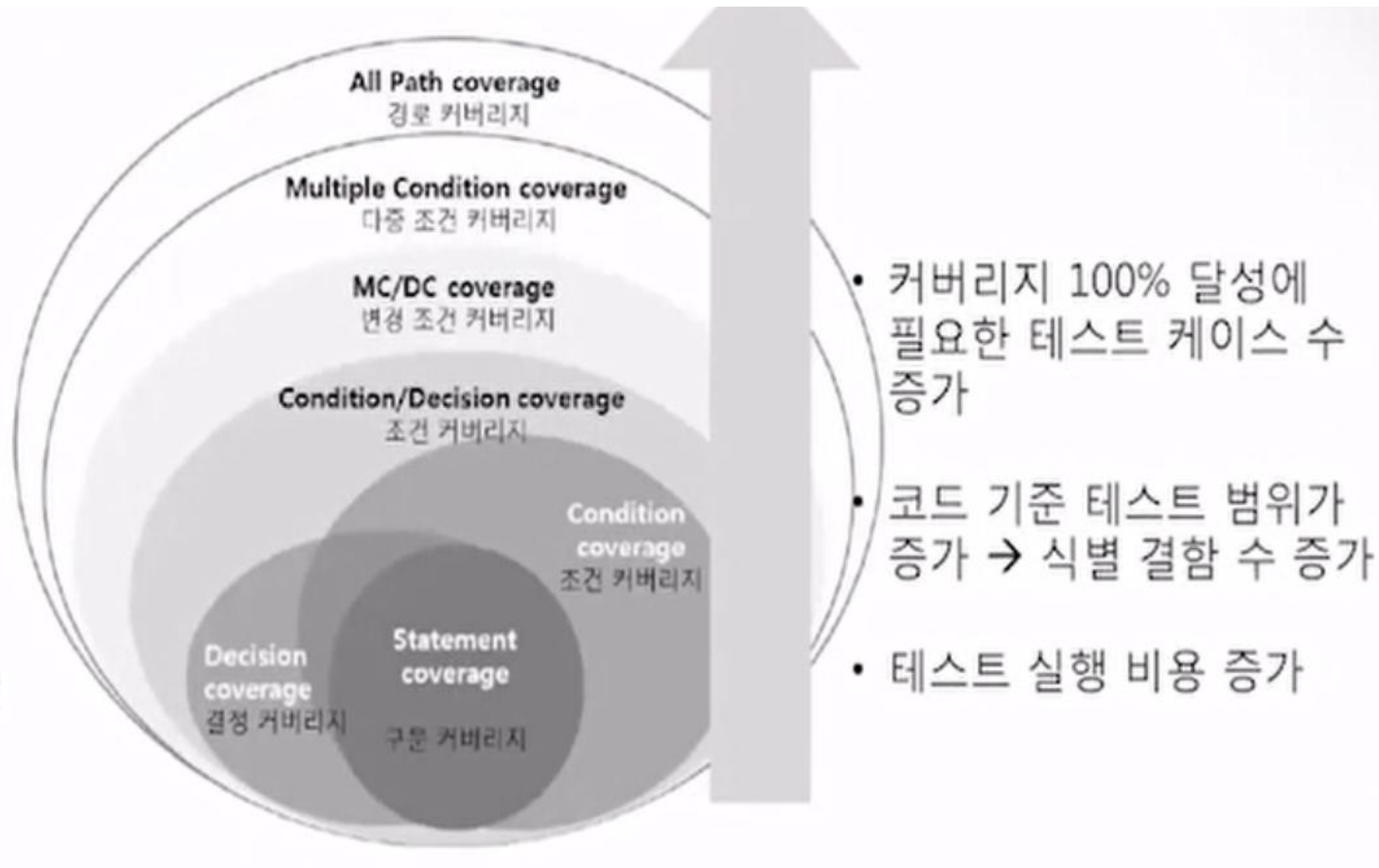
	(X, Y, Z)	(X==1)	(Y==1)	(Z==1)	(X==1) OR (Y==1 AND Z==1)		
1	(1,1,1)	T	T	T	T		
2	(1,1,0)	T	T	F	T	x1	V
3	(1,0,1)	T	F	T	T	x2	
4	(1,0,0)	T	F	F	T	x3	
5	(0,1,1)	F	T	T	T	y1 z1	V
6	(0,1,0)	F	T	F	F	x1 z1	V
7	(0,0,1)	F	F	T	F	x2 y1	V
8	(0,0,0)	F	F	F	F	x3	

→ (2,5,6,7) 또는 (3,5,6,7)





4. 설계 기법- 커버리지간 포함 관계

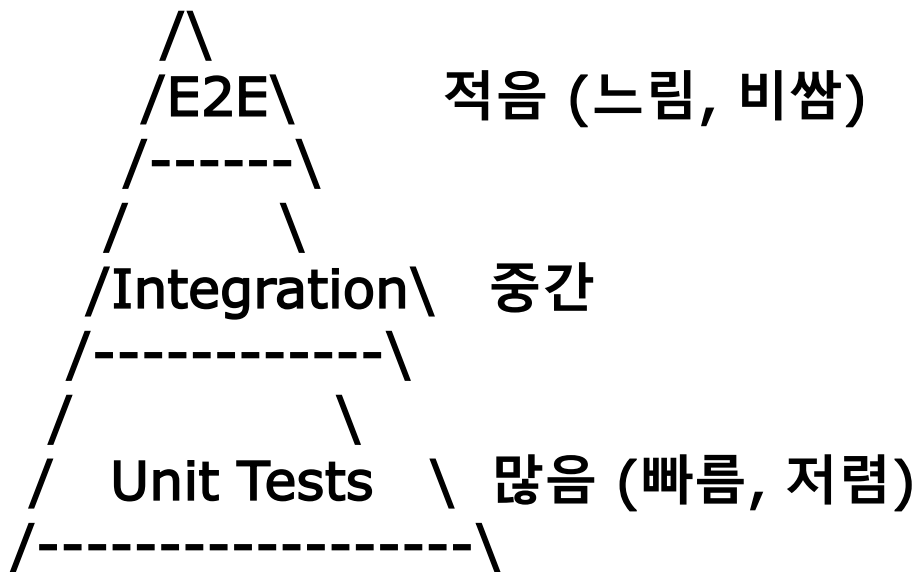




4. 테스트 케이스 설계 기법

테스트 피라미드

장점: ````



OrderBean 적용 예시

Unit: calculate_discount() 함수 테스트

Integration: API 엔드포인트 + DB 통합 테스트

E2E: 전체 주문 플로우



4. 테스트 케이스 설계 기법



| 정상 케이스 (Happy Path)

```
def test_create_order_success():  
    # 모든 입력이 올바른 경우
```

| 경계값 테스트 (Boundary)

```
def test_order_with_zero_quantity():  
def test_order_with_max_items():
```



4. 테스트 케이스 설계 기법



| 예외 케이스 (Exception)

```
def test_order_with_invalid_coupon():  
def test_order_with_expired_coupon():
```

| 조합 테스트

```
def test_order_with_multiple_coupons_and_items():
```



4. 테스트 케이스 설계 기법



| 예외 케이스 (Exception)

```
def test_order_with_invalid_coupon():  
def test_order_with_expired_coupon():
```

| 조합 테스트

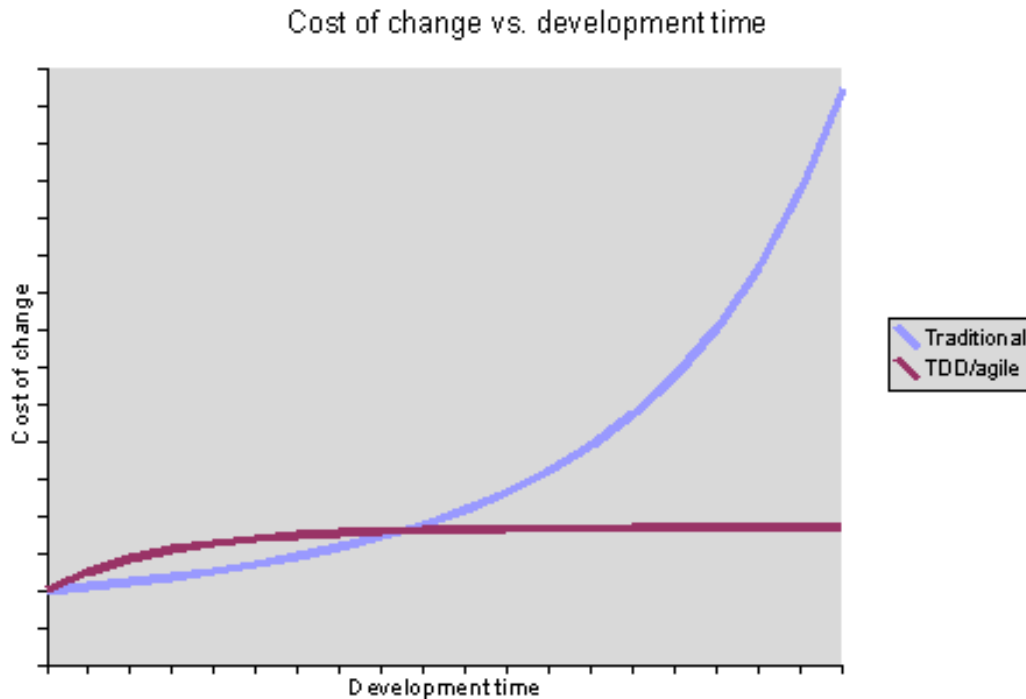
```
def test_order_with_multiple_coupons_and_items():
```

5. 테스트 종류

테스트 자동화

TDD : 15~35% 의 개발시간 증가, 40~90% 의 결함률 감소

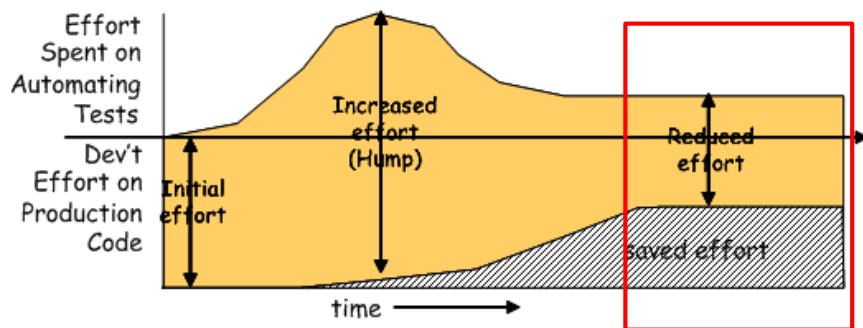
- 출처 : Microsoft Exploding Software-Engineering Myths By Janie Chang



5. 테스트 종류

테스트 자동화

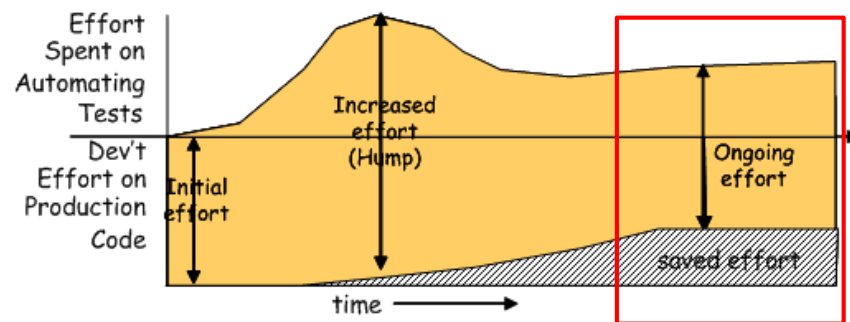
- 자동화 테스트 적용에 대한 높은 장벽
- 잘 적용된 자동화 테스트 안정화 -> Production 개발에 대한 노력 감소
- 테스트 작성이 어렵고 유지보수가 높은 자동화 테스트는 지속적인 노력을 필요로 함.
- 단순히 자동화 테스트를 도입하는 것이 중요한 것이 아니라, "올바르게 자동화하는 것"이 ROI를 높이는 핵심이다!



Sketch Economics-Good embedded from Economics-Good.gif

Fig. X: An automated unit test project with a good ROI.

The cost-benefit trade off where the total cost is reduced by good test practices.



Sketch Economics-Bad embedded from Economics-Bad.gif

Fig. X: An automated unit test project with a poor ROI.

The cost-benefit trade off where the total cost is increased by poor test practices.

Test Automation Manifesto

- Concise – As simple as possible and no simpler.
- Self Checking – Test reports its own results ; needs no human interpretation.
- Repeatable – Test can be run many times in a row without human intervention.
- Robust – Test produces same result now and forever.
- Tests are not affected by changes in the external environment.
- Sufficient – Tests verify all the requirements of the software being tested.
- Necessary – Everything in each test contributes to the specification of desired behavior.
- Clear – Every statement is easy to understand.
- Efficient – Tests run in a reasonable amount of time.
- Specific – Each test failure points to a specific piece of broken functionality; unit test failures provide “defect triangulation”.
- Independent – Each test can be run by itself or in a suite with an arbitrary set of other tests in any order.
- Maintainable – Tests should be easy to understand and modify and extend.
- Traceable – To and from the code it tests and to and from the requirements.

6. 테스트 케이스

| 테스트 케이스(Test Case)의 개념

- 테스트 케이스란 요구 사항을 준수하는지 검증하기 위하여 테스트 조건, 입력값, 예상 출력값 및 수행한 결과 등 테스트 조건을 명시한 것이다.
- 테스트 케이스는 테스트 대상인 특정 프로그램 경로의 실행, 요구사항과 일치 여부 확인을 위해 디자인된 입력값의 묶음, 실행 사전 조건, 기대 결과와 실행 사후 조건의 집합이다. 즉 테스트 되어야 하는 특정 조건+입력값+기대 결과를 포함한다.

6. 테스트 케이스

I 구성요소

- 테스트 케이스는 테스트 조건을 확인하기 위해 개발한다. 테스트 케이스는 고객의 요구사항과 요구사항과의 일치성을 확인하고, 요구사항과의 관계를 확인하기 위해 연관 관계 정보를 확보하여야 한다.
- 테스트 케이스 구성 요소(IEEE 829)

항목	설명
식별자 (Identifier)	항목 식별자, 일련번호
테스트 항목 (Test Item)	테스트할 모듈 또는 기능
입력 명세 (Input Specification)	입력값 또는 테스트 조건
출력 명세 (Output Specification)	테스트 케이스 실행 시 기대되는 출력값 결과
환경 설정 (Environmental Needs)	테스트 수행 시 필요한 하드웨어나 소프트웨어 환경
특수 절차 요구 (Special Procedure Requirement)	테스트 케이스 수행 시 특별히 요구되는 절차
의존성 기술 (Inter-case Dependencies)	테스트 케이스 간의 의존성



테스트 케이스 예

테스트 케이스			프로젝트명		인사관리 앱 시스템 구축		
			대상 시스템		정산 시스템		
단계	단위 테스트	작성자	홍길동	승인자	박문수	문서 상태	최초 작성
작성일	2020-09-01	버전	v1.0	테스트 범위	공통 모듈	테스트 조직	개발팀
테스트 ID	TC-CMM-001("Common Module" or "Core Mathematical Module") TC-AO-001 ("Arithmetic Operations")			테스트 일자	2020-09-01		
테스트 목적	사칙연산 정확도 테스트						
테스트 기능	+ 기능 점검						
입력값	1,10 / 0,1/ 0,0/ 0,-1/-1,-10						
테스트단계	테스트 케이스			예상값	중요도	성공 /실패	비고
	1+10			11	중요	성공	
	0+1			1	중요	성공	
	0 / 0			예외 발생	중요	성공	
	추가된 테스트 사례						
	1. 예외 처리 확인: • 0 / 0 → ArithmeticException 예외 발생 확인. 다양한 조합:						
테스트환경	자바 JDK 버전: 17 IDE: IntelliJ IDEA 2023.2 운영 체제: 윈도우 10						
전제 조건	프로그래밍은 오류 없이 성공적으로 컴파일되어야 합니다. 모든 종속성을 올바르게 설치하고 구성해야 합니다.						
성공 /실패 기준	성공: 모든 테스트 사례가 예상한 결과를 생성합니다. 실패: 테스트 케이스가 예상한 결과를 생성하지 않습니다.						
특별하게 필요한 절차	테스트 결과를 기록하고 이에 따라 테스트 사례 문서를 업데이트합니다. 즉각적인 해결을 위해 모든 실패를 개발팀에 전달하세요.						

6. 테스트 케이스

테스트 케이스 목록 양식

ID	구분	사전조건	수행절차	기대결과	결과	추적성	중요도	비고
1.1	우편번호 조회	동 이름, 번지 등 5자 이상 입력 후 조회 버튼 클릭	1. 주소 변경 페이지 이동 2. 주소 페이지에서 우편번호 조회 아이콘 클릭	팝업 윈도우 표시 후 조회 데이터 출력 조회된 내용이 없는 경우 데이터가 없다는 내용의 메시지 제공		요구사항 ID-100	중	우편번호로 주소 조회 가능

6. 테스트 케이스

테스트 케이스 작성 시 견지할 수 있는 지침

- OKR (Objectives and Key Results)
 - "목표(Objective)는 영감을 주고, 핵심 결과(Key Results)는 실행을 이끈다."
 - 넓게 보는 비전(목표)을 설정하면서도 이를 달성하기 위한 실행 가능한 구체적인 지표(핵심 결과)를 설정하는 방법론.
 - Google 등에서 성공적으로 활용된 사례.

사례 1: Gmail 출시

Objective: Gmail을 성공적으로 런칭하기.

Key Results:

베타 사용자 100만 명 확보.

평균 이메일 전송 속도를 0.1초 이하로 줄이기.

사용자 피드백 점수 90점 이상 달성.

사례 2: Google Chrome

Objective: 세계에서 가장 빠르고 안정적인 브라우저를 만들기.

Key Results:

브라우저 로딩 속도를 3초 이하로 줄이기.

사용자 만족도 점수 95% 이상 유지.

6개월 내 시장 점유율 10% 도달.

6. 테스트 케이스

테스트 목표 설정

- SMART 원칙에 따라 목표를 수립한다. 목표는 다음처럼 구체적, 측정

구분	설명
specific	• 목표는 구체적이어야 한다.
measurable	• 목표는 측정 가능해야 한다.
attainable	• 목표는 달성 가능한 수준이어야 한다.
realistic	• 목표는 현실적이어야 한다.
time based	• 목표는 시간 제한을 두고 설정해야 한다.

6. 테스트 케이스

테스트 목표 설정

1. 비즈니스 모델을 구축한다.
2. 대상 운영 플랫폼에 대한 품질 차원을 분석하고 테스트 기법을 적용한다.
3. 품질에 대한 이해를 통하여 목표를 설정한다.

● 테스트 목표 수립에 대한 예

품질목표	평가 항목	목표값	평가 기준
적합성	기능 구현 완전성	100%	전체 기능에 대한 테스트 케이스 시험
정확성	기대 정확도	100%	전체 기능에 대한 테스트 케이스 시험
	정밀도	100%	계산을 수행하는 테스트 케이스 시험
신뢰성	잔존 결함률	낮은 심각도: 20% 이하 높은 심각도: 100%	발견된 결함 중 심각도에 따라 수정된 정도
효율성	응답 시간 적합성	80% 이상	시스템 동시 접속자 1,000명에서 페이지 로딩 속도가 3초 이내
	메모리 오류	0%	메모리 오류가 발생하면 안 됨.

6. 테스트 케이스

테스트 목표 설정

테스트 대상의 품질수준 예시

품질특성	평가항목	목표 값	평가기준(적합기준)	평가결과
기능성	기능구현 완전성	X $\approx$ 100%	전체기능 TC중 100%성공	최종조치결과 반영
	계산 정확도	X $\approx$ 95%	계산을 수행하는 TC중 95% 이상이 대략 정확한 값 도출	목표값 만족
	데이터 교환성	X $\approx$ 95%	다른 소프트웨어나 시스템과 데이터 교환을 시도한 전체 TC 중 95% 이상이 대한 교환 성공	목표값 만족
보안성	접근 제어성	X $\approx$ 0%	규정된 불법 조작은 성공한 불법조작이나, 규정되지 않은 불법조작이 없음	목표값 만족
	사용자 인증성	X $\approx$ 100%	전체사용자 중 사전에 허가된 사용자만 통과	목표값 만족
신뢰성	결함밀도	X $\leq$ 3%	시스템 테스트 시 전체 수행된 TC 중 결함된 결함율이 3% 이하	최종조치결과 반영
사용성	기능이해 가능성	X $\approx$ 90%	인터뷰스로 가용한 기능 중 모든 기능에 대하여 사용자가 목적을 바르게 기술한 기능이 90% 이상	목표값 만족
효율성	평균 응답시간	X $\leq$ 1	실제 응답시간의 평균이 규정된 평균응답시간($\leq$ 5sec)이하	목표값 만족
	평균처리량	X $\geq$ 1	실제 작업 처리량의 평균이 규정된 평균 작업 처리량 이상	목표값 만족
유지보수성	결함 분석능력	X $\geq$ 95%	결함된 전체 TC 중 원인을 파악하지 못한 고장TC가 5% 미만(95%이상의 고장 TC에 대한 원인파악)	목표값 만족
이식성	이식성	X $\approx$ 100%	상이한 디바이스 운영체제 및 브라우저에서 제대로 동작 가능	목표값 만족
상호운용성	상호운용성	X $\approx$ 100%	유사App을 같은 기기에 설치하여 충돌이 발생하지 않고 제대로 동작 가능	목표값 만족

Pytest란?

Pytest는 파이썬 코드를 테스트하기 위한 프레임워크로, 단위, 기능, 통합 테스트를 모두 지원하며 간단하고 직관적인 문법을 사용합니다. ``test_``로 시작하는 파일 및 함수 이름을 지키면 테스트가 자동으로 탐지되고, `assert` 문만으로 간단하게 테스트를 작성할 수 있다는 특징이 있습니다. 또한, `fixtures`와 같은 강력한 확장 기능을 통해 테스트 환경 설정을 효율적으로 관리할 수 있습니다.



7. Pytest

주요 특징

- 자동 테스트 탐지: ``test_``로 시작하는 파일명과 함수명을 사용하면 별도 설정 없이 테스트가 자동으로 인식됩니다.
- 간단한 문법: 복잡한 클래스 선언 없이 `assert` 문만으로 테스트 결과를 확인할 수 있습니다.
- 다양한 테스트 지원: 단위 테스트, 기능 테스트, 통합 테스트 등 다양한 종류의 테스트를 지원합니다.
- 확장 가능한 기능: `fixtures`와 같은 기능을 통해 데이터베이스 설정 등 복잡한 테스트 환경을 재사용 가능하게 설정할 수 있습니다.
- 유연성: Python의 내장 `unittest`로 작성된 테스트도 실행할 수 있는 등 유연성이 높습니다.

테스트 작성

1. 테스트 파일: `test_`로 시작하거나 `_test.py`로 끝나는 파일에 테스트 코드를 작성합니다.
2. 테스트 함수: `test_`로 시작하는 함수를 작성합니다.
3. 테스트 코드: `assert` 문을 사용하여 예상 결과와 실제 결과를 비교합니다.

● 테스트 실행

- 터미널에서 프로젝트 루트 디렉토리로 이동한 후 `pytest` 명령어를 입력합니다.
- `pytest`는 `test_`로 시작하는 파일과 함수를 자동으로 찾아 실행하고 결과를 보여줍니다.



7. Pytest

기본 구조

```
import pytest
from app.services import OrderService
```

```
@pytest.fixture
def order_service():
    """테스트용 서비스 인스턴스"""
    return OrderService()
```

```
@pytest.fixture
def sample_user():
    """테스트용 사용자"""
    return User(id=1, name="Test User", email="test@example.com")
```

```
def test_calculate_subtotal(order_service, sample_user):
    items = [OrderItem(price=5000, quantity=2)]
    result = order_service.calculate_subtotal(items)
    assert result == 10000
```



8. FastAPI

| FastAPI란 무엇인가?

- FastAPI는 Python 3.7+ 기반의 비동기(Asynchronous) 웹 프레임워크로, 빠른 속도(Fast)와 쉬운 개발(Easy), 자동 문서화(Smart Docs)를 모두 갖춘 차세대 백엔드 웹 프레임워크입니다.



8. FastAPI

FastAPI의 탄생 배경

- FastAPI는 Flask의 단순함 + Django의 안정성 + Node.js 수준의 속도를 모두 결합한 “Python계의 Express.js”라고 이해할 수 있습니다.

기존 프레임워크	특징	FastAPI의 개선점
Flask	간단하지만 확장 어려움	기본기능 내장 (Validation, Docs 등)
Django	거대한 생태계, 그러나 무겁고 느림	경량 + 비동기 + REST 중심
Tornado / Sanic	빠르지만 문법 난해	간결한 문법 + 타입 기반 자동화

8. FastAPI

| 왜 FastAPI인가?

- “FastAPI = Python으로 만드는 현대적인, 빠르고 안전한 REST API 표준”

비교 항목	Flask	Django	FastAPI
속도	느림	중간	⚡ 빠름
문서 자동화	❌ 없음	❌ 없음	✅ Swagger 지원
타입 힌트 기반 검증	❌	❌	✅
비동기 지원	❌	제한적	✅ 완전 지원
학습 곡선	쉬움	보통	쉬움
테스트 용이성	보통	어려움	✅ 매우 높음



8. FastAPI

FastAPI 사용법

● 설치

- 먼저 프로젝트 폴더를 만들고, 가상 환경을 생성하고 활성화합니다.

```
C:\DEV\SamsungDisplay\Week_2\Code_pro>mkdir fastapi
```

```
C:\DEV\SamsungDisplay\Week_2\Code_pro>cd fastapi
```

```
C:\DEV\SamsungDisplay\Week_2\Code_pro\fastapi>python -m venv .venv
```

```
C:\DEV\SamsungDisplay\Week_2\Code_pro\fastapi>.venv\Scripts\activate
```

- 가상 환경이 활성화된 상태에서 pip를 사용하여 FastAPI와 실행 도구인 uvicorn을 설치합니다.

```
pip install fastapi uvicorn
```

```
(.venv) C:\DEV\SamsungDisplay\Week_2\Code_pro\fastapi>pip install  
fastapi uvicorn
```

```
(.venv) C:\DEV\SamsungDisplay\Week_2\Code_pro\fastapi>pip list
```

```
(.venv) C:\DEV\SamsungDisplay\Week_2\Code_pro\fastapi>pip freeze > requirements.txt
```



8. FastAPI

FastAPI 사용법

- 기본 애플리케이션 만들기

- FastAPI 애플리케이션 인스턴스를 생성합니다.

```
from fastapi import FastAPI
```

```
app = FastAPI()
```

- `@app.get("/")`와 같은 데코레이터를 사용하여 API 엔드포인트(경로)를 정의합니다. 이는 GET 메서드로 루트 경로에 접근할 때 실행될 함수를 지정합니다.

```
@app.get("/")
```

```
async def read_root():
```

```
    return {"Hello": "World"}
```

8. FastAPI

FastAPI의 기본 구조

```
# main.py
from fastapi import FastAPI

app = FastAPI(title="OrderBean API")

@app.get("/")
def root():
    return {"message": "OrderBean backend running !"}

@app.get("/menu/{item_id}")
def get_menu(item_id: int):
    return {"item_id": item_id, "name": "Americano", "price": 4500}
```



FastAPI 사용법

- 애플리케이션 실행

- 터미널에서 uvicorn 명령어를 사용하여 서버를 실행합니다.
- main은 애플리케이션이 정의된 파이썬 파일 이름이고, app은 FastAPI 인스턴스 변수입니다.

uvicorn main:app --reload

- 이제 웹 브라우저에서 <http://127.0.0.1:8000>으로 접속하면 {"Hello": "World"}가 표시되는 것을 확인할 수 있습니다.

```
.venv) C:\DEV\SamsungDisplay\Week_2\Code_pro\fastapi>uvicorn main:app --reload
INFO: Will watch for changes in these directories: ['C:\\DEV\\SamsungDisplay\\Week_2\\Code_pro\\fastapi']
INFO: Uvicorn running on http://127.0.0.1:8000 (Press CTRL+C to quit)
INFO: Started reloader process [8716] using StatReload
```

← → ↻ ⓘ 127.0.0.1:8000

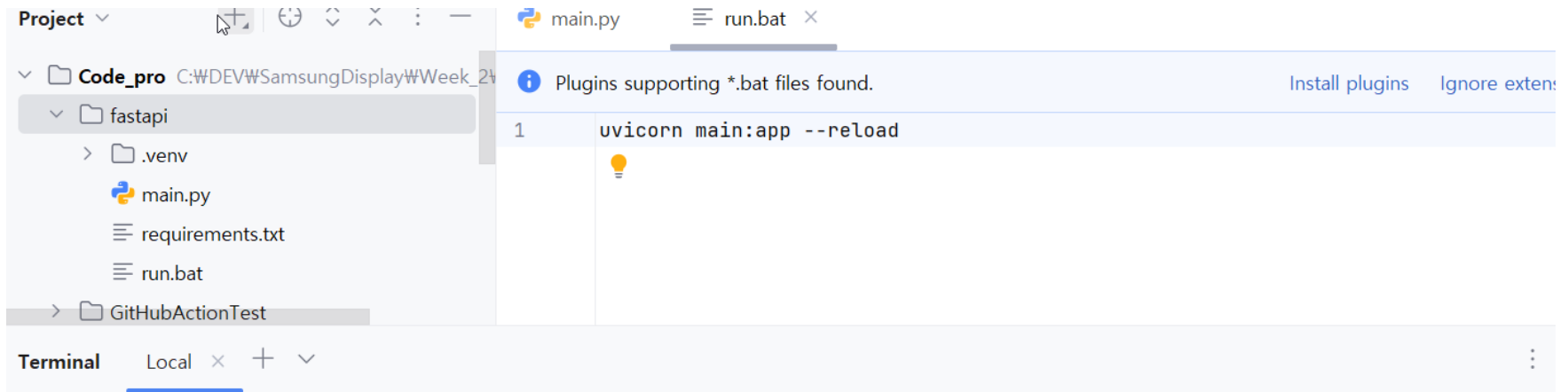
retty print 적용 ☐

```
"message": "OrderBean backend running !"}>
```

8. FastAPI

FastAPI 사용법

- run.bat 파일 만들어 실행해 보기



```
(.venv) C:\DEV\SamsungDisplay\Week_2\Code_pro\fastapi>run
```

```
(.venv) C:\DEV\SamsungDisplay\Week_2\Code_pro\fastapi>uvicorn main:app --reload
```

```
INFO:      Will watch for changes in these directories: ['C:\\DEV\\SamsungDisplay\\Week_2\\Code_pro\\fastapi']
```

```
INFO:      Uvicorn running on http://127.0.0.1:8000 (Press CTRL+C to quit)
```

FastAPI 사용법

- 다른 HTTP 메서드 사용하기
 - GET, POST, PUT 등 다양한 HTTP 메서드를 사용해 API를 만들 수 있습니다.
 - `@app.post("/items/")`와 같이 `post` 데코레이터를 사용하면 POST 요청을 처리하는 API 엔드포인트를 만들 수 있습니다.
- 자동 API 문서 확인하기
 - FastAPI는 Swagger UI 및 ReDoc을 통해 자동으로 API 문서를 생성합니다.
 - 웹 브라우저에서 `http://127.0.0.1:8000/docs`에 접속하여 API 문서를 확인할 수 있습니다.

OrderBean API 0.1.0 OAS 3.1

/openapi.json

default

GET / Root

GET /menu/{item_id} Get Menu

Schemas

HTTPValidationError > Expand all object

ValidationError > Expand all object

FastAPI의 핵심 기능 상세

(1) 자동 문서화

FastAPI는 OpenAPI 표준 기반으로,
모든 엔드포인트를 자동으로 Swagger 문서로 생성합니다.

```
@app.post("/order/")
def create_order(order: dict):
    return {"status": "success", "order": order}
```

→ /docs 페이지에서 자동으로 다음이 표시됩니다 👉

HTTP 메서드 (POST)

요청 Body 구조

응답 JSON 스키마

FastAPI의 핵심 기능 상세

(2) 유효성 검증(Validation)

FastAPI는 Pydantic 모델을 이용해 입력 데이터를 자동 검증합니다.

```
from pydantic import BaseModel

class MenuItem(BaseModel):
    name: str
    price: int
    is_available: bool = True

@app.post("/menu/")
def create_menu(item: MenuItem):
    return {"message": "메뉴 등록 완료", "data": item}
```

잘못된 타입 입력 시 자동 에러 반환:

```
{
  "detail": [
    {
      "loc": ["body", "price"],
      "msg": "value is not a valid integer",
      "type": "type_error.integer"
    }
  ]
}
```

FastAPI의 핵심 기능 상세

(3) 비동기(async) 지원

FastAPI는 Python의 `async/await` 문법을 완벽히 지원합니다. I/O 작업(DB, API 호출 등)을 병렬로 처리하여 속도를 극대화합니다.

```
import asyncio

@app.get("/simulate")
async def simulate_task():
    await asyncio.sleep(2)
    return {"message": "2초 후 응답"}
```

FastAPI의 핵심 기능 상세

(4) 보안 / 인증 기능 내장

FastAPI는 OAuth2 / JWT 기반 인증을 내장 지원합니다.

```
from fastapi.security import OAuth2PasswordBearer

oauth2_scheme = OAuth2PasswordBearer(tokenUrl="token")

@app.get("/users/me")
async def read_users_me(token: str = Depends(oauth2_scheme)):
    return {"token": token}
```

Swagger UI에서 자동으로 “Authorize” 버튼 생성 → 토큰 입력 테스트 가능.

FastAPI의 핵심 기능 상세

(5) 테스트 친화적 구조

FastAPI는 TestClient로 API 테스트를 쉽게 작성할 수 있습니다.

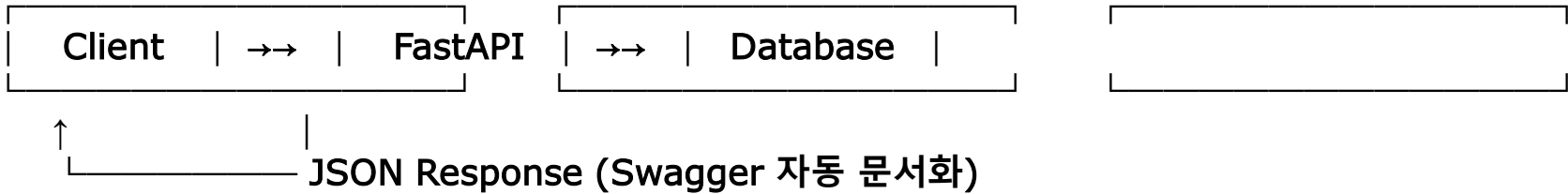
```
from fastapi.testclient import TestClient
from app.main import app
```

```
client = TestClient(app)
```

```
def test_root():
    res = client.get("/")
    assert res.status_code == 200
    assert res.json() == {"message": "OrderBean backend running 🚀"}
```

8. FastAPI

FastAPI의 실행 구조 (요청-응답 흐름)



처리 단계 요약:

1. 클라이언트가 API 요청을 보냄
2. FastAPI가 요청을 Pydantic 모델로 검증
3. 비즈니스 로직 처리 (DB / 외부 API 호출)
4. JSON 응답 반환
5. 자동 문서 업데이트

8. 테스트 커버리지

| 커버리지 측정

```
pytest --cov=app --cov-report=html
```

목표 설정

Critical Path: 95%+ (결제, 인증)

Business Logic: 85%+ (주문, 할인 계산)

Utils/Helpers: 70%+

전체: 80%+

⚠ 주의: 커버리지 100%가 목표가 아니다!

→ 의미 있는 테스트인지가 중요

8. 테스트 커버리지

| 기능에 대한 테스트 커버리지 위험 테스트 커버리지 예시

위험 아이템	우선순위	장애 가능성	예상 손실	테스트 케이스	실행 여부
위험 아이템 1	1	9	데이터 손실	TC_PER_01 TC_CM_01 TC_TB_02	O O O
위험 아이템 1	1	9	손해 배상		X
위험 아이템 1	2	7	업무 중단	TC_UI_04 TC_TB_04	O X
...					
위험 아이템 n	5	1	사용자 불편	TC_UI_01 TC_TB_01	O



9. 테스트 더블 (Test Doubles)

Test Double

- When we are writing a test in which we cannot (or chose not to) use a real depended-on component (DOC), we can replace it with a Test Double.
- The Test Double doesn't have to behave exactly like the real DOC; it merely has to provide the same API as the real one so that the SUT thinks it is the real one!

By Gerad Meszaros(xUnit Test Patterns 저자 저라드 메스자로스) -

- 영화 촬영에서 스텐트 대역 배우를 의미하는 스텐트 더블에서 아이디어를 얻어 만들어지 용어
- 테스트 더블은 의존 구성요소를 사용할 수 없을 때 테스트 대상 코드와 상호작용 하는 객체

9. 테스트 더블 (Test Doubles)

| 언제 사용하는가?

- Object 가 비결정적 결과를 Return 할 때 (현재 시간, 온도 등)
- 재현/구현하기 힘든 상태를 가질 때 (Network error 등)
- 동작이 느릴 때
- 아직 구현되지 않았거나 동작이 바뀌었을 때
- Test 를 위한 method 나 information 을 포함해야 할 때
- Test 를 위해 호출시 parameter 나 호출 횟수 확인이 필요



9. 테스트 더블 (Test Doubles)

종류

용어	설명
Dummy	아무것도 하지 않지만, 인자를 채우기 위해 존재함
Fake	실제 구현이지만, 간단하게 동작하도록 대체한 객체 (예: 인메모리 DB)
Stub	특정한 상황에서 미리 정의된 값을 반환
Spy	실제 객체를 감싸서 동작을 기록하고 일부는 조작 가능
Mock	행위를 검증하기 위해 만든 가짜 객체

구분	역할	실제 행위	결과 조작	행위 검증
Dummy	전달용	X	X	X
Fake	실제 동작	O	X	△
Stub	결과 반환	X	O	X
Spy	일부 진짜	O	△	O
Mock	모두 가짜	X	O	O



9. 테스트 더블 (Test Doubles)



Mock: 외부 의존성 대체

```
from unittest.mock import Mock
```

```
def test_order_sends_email(mock):  
    mock_email = mock.patch('app.services.send_email')  
  
    create_order(user_id=1, items=[...])  
  
    mock_email.assert_called_once()
```



9. 테스트 더블 (Test Doubles)

Fake: 간단한 구현체

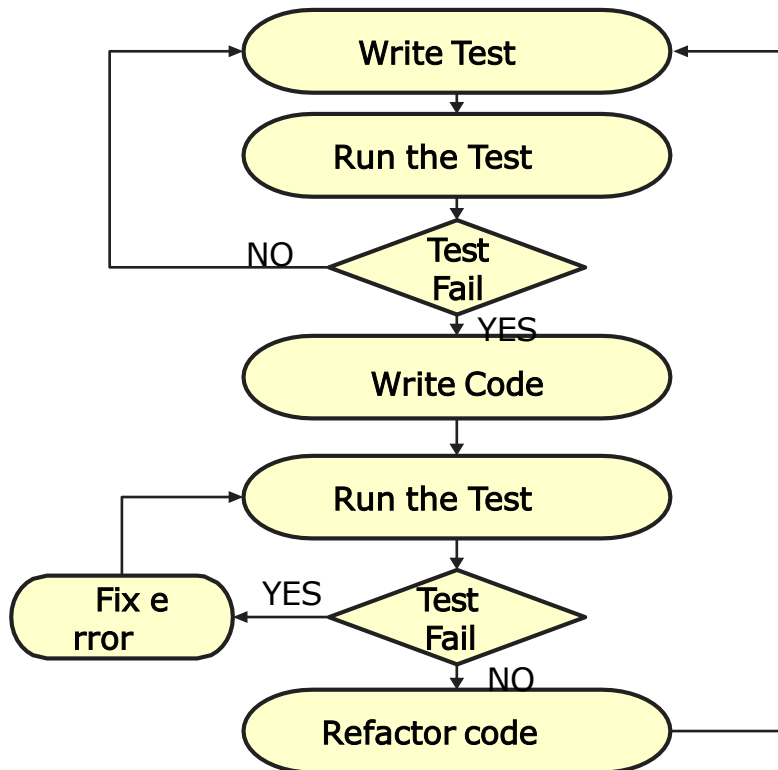
```
# SQLite in-memory DB for testing  
SQLALCHEMY_DATABASE_URL = "sqlite:///memory:"
```

10. TDD 사이클 (Red-Green-Refactor)

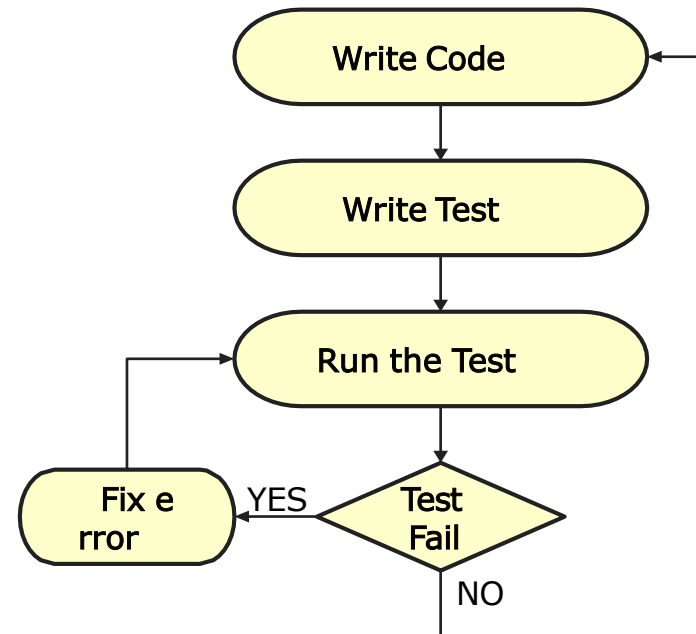
TDD

Test the program before you write it.
-TDD by Example, Kent Beck

TDD



TLD(Test Last Development)



10. TDD 사이클 (Red-Green-Refactor)

TDD 세 가지 법칙

실패하는 단위 테스트를 작성할 때까지 실제 코드를 작성하지 않는다.

컴파일은 실패하지 않으면서 실행이 실패하는 정도로만 단위 테스트를 작성한다.

현재 실패하는 테스트를 통과할 정도로만 실제 코드를 작성한다.

- 출처 : Clean Code, Robert C Martin

10. TDD 사이클 (Red-Green-Refactor)

TDD 세 가지 법칙

1. RED ❌ 실패하는 테스트 작성



2. GREEN ✅ 최소한의 코드로 통과



3. REFACTOR 🔧 코드 개선



반복...

10. TDD 사이클 (Red-Green-Refactor)

TDD 세 가지 법칙

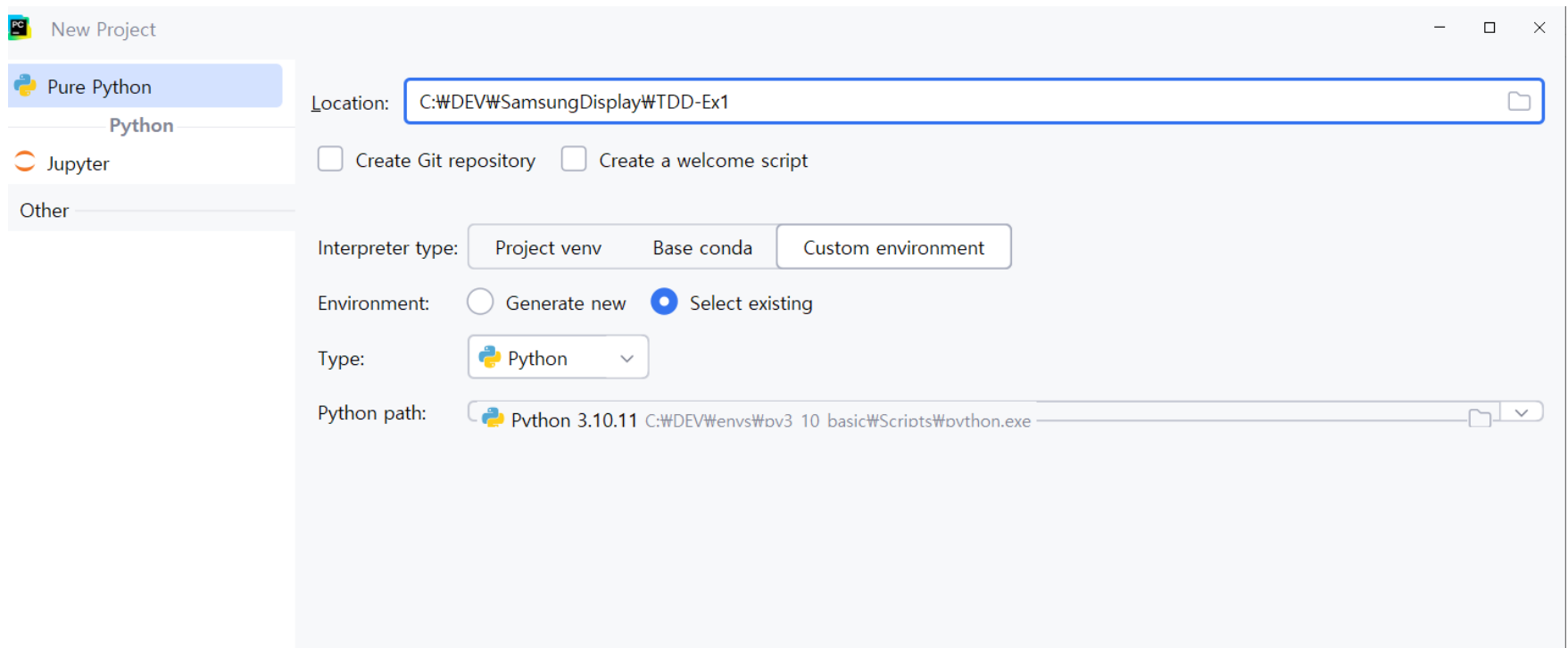
TDD 및 Refactoring을 이용한 Clean Code 달성

1. 질문(Ask): 테스트 작성을 통해 시스템에 질문한다. (테스트 수행 결과는 실패)
2. 응답(Respond): 테스트를 통과하는 코드를 작성해서 질문에 대답한다. (테스트 성공)
3. 정제(Refine, Refactoring): 아이디어를 통합하고, 불필요한 것은 제거하고, 모호한 것은 명확히 해서 대답을 정제한다. (리팩토링)
4. 반복(Repeat): 다음 질문을 통해 대화를 계속 진행한다.

10. TDD 사이클 (Red-Green-Refactor)

TDD(Test-Driven Development)와 Refactoring을 통해 Clean Code를 달성하는 과정

예제: 숫자 더하기 계산기 실습



10. TDD 사이클 (Red-Green-Refactor)

TDD(Test-Driven Development)와 Refactoring을 통해 Clean Code를 달성하는 과정

예제: 숫자 더하기 계산기

1. 질문(Ask): 테스트 작성

- 먼저 시스템에 질문하는 테스트 코드를 작성합니다. 이 테스트는 현재 코드가 존재하지 않는 상태에서 실패하게 됩니다.
- 의도:
- "2 + 3은 5가 되어야 한다"는 요구사항을 테스트로 표현한 것.

test_calculator.py

```
import pytest
from calculator import Calculator

def test_add_two_numbers():
    calc = Calculator()
    result = calc.add(2, 3)
    assert result == 5 # 기대: 2 + 3 = 5
```

테스트 실행

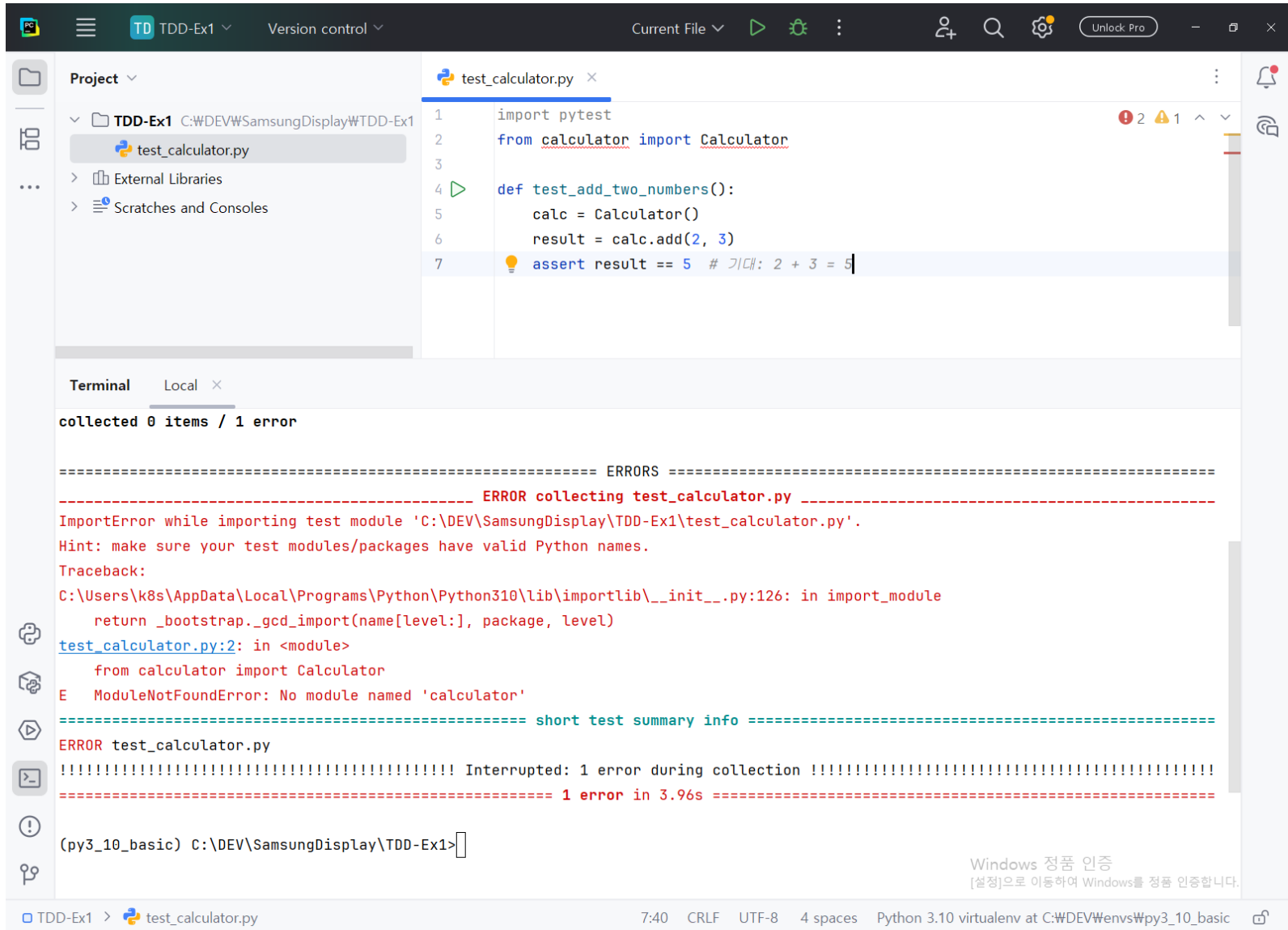
pytest -v

▼ 예상 결과:

E ModuleNotFoundError: No module named 'calculator'

👉 실패(Fail) = 정상입니다.
아직 Calculator 클래스가 없으니 RED 단계에 해당합니다.

10. TDD 사이클 (Red-Green-Refactor)



The screenshot shows a code editor with a project named 'TDD-Ex1'. The file 'test_calculator.py' is open, showing the following code:

```
1 import pytest
2 from calculator import Calculator
3
4 def test_add_two_numbers():
5     calc = Calculator()
6     result = calc.add(2, 3)
7     assert result == 5 # 기대: 2 + 3 = 5
```

The terminal output shows the following error:

```
collected 0 items / 1 error

===== ERRORS =====
----- ERROR collecting test_calculator.py -----
ImportError while importing test module 'C:\DEV\SamsungDisplay\TDD-Ex1\test_calculator.py'.
Hint: make sure your test modules/packages have valid Python names.
Traceback:
C:\Users\k8s\AppData\Local\Programs\Python\Python310\lib\importlib\__init__.py:126: in import_module
    return _bootstrap._gcd_import(name[level:], package, level)
test_calculator.py:2: in <module>
    from calculator import Calculator
E   ModuleNotFoundError: No module named 'calculator'

===== short test summary info =====
ERROR test_calculator.py
!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!! Interrupted: 1 error during collection !!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!
===== 1 error in 3.96s =====

(py3_10_basic) C:\DEV\SamsungDisplay\TDD-Ex1>
```

The status bar at the bottom indicates the current file is 'test_calculator.py' and the environment is 'Python 3.10 virtualenv at C:\DEV\envs\py3_10_basic'.

10. TDD 사이클 (Red-Green-Refactor)

TDD(Test-Driven Development)와 Refactoring을 통해 Clean Code를 달성하는 과정

예제: 숫자 더하기 계산기

- 2. 응답(Respond): 테스트를 통과하는 최소한의 코드 작성
- 테스트를 통과하기 위해 최소한의 코드를 작성합니다.

calculator.py

```
class Calculator:  
    def add(self, a, b):  
        return a + b
```

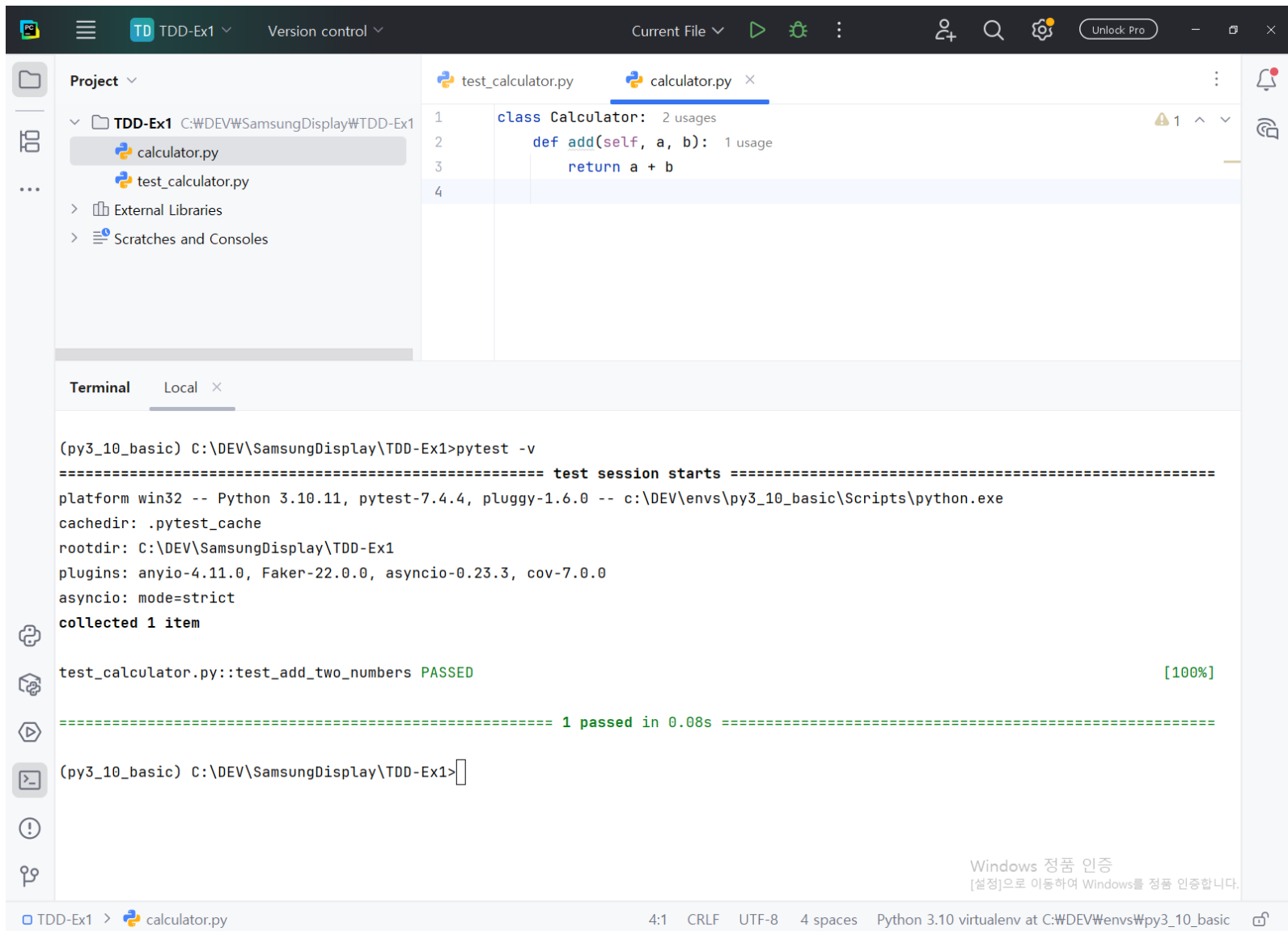
다시 실행
pytest -v

✅ 결과:

```
collected 1 item  
test_calculator.py::test_add_two_numbers PASSED  
[100%]
```

테스트 통과 🎉
즉, 질문에 대한 “정답”을 코드로 작성했습니다.

10. TDD 사이클 (Red-Green-Refactor)



The screenshot displays the Visual Studio Code interface during a TDD cycle. The editor shows the `calculator.py` file with the following code:

```
1 class Calculator:
2     def add(self, a, b):
3         return a + b
```

The terminal window shows the execution of `pytest -v` in a virtual environment. The output indicates that the test passed successfully.

```
(py3_10_basic) C:\DEV\SamsungDisplay\TDD-Ex1>pytest -v
===== test session starts =====
platform win32 -- Python 3.10.11, pytest-7.4.4, pluggy-1.6.0 -- c:\DEV\envs\py3_10_basic\Scripts\python.exe
cachedir: .pytest_cache
rootdir: C:\DEV\SamsungDisplay\TDD-Ex1
plugins: anyio-4.11.0, Faker-22.0.0, asyncio-0.23.3, cov-7.0.0
asyncio: mode=strict
collected 1 item

test_calculator.py::test_add_two_numbers PASSED [100%]

===== 1 passed in 0.08s =====

(py3_10_basic) C:\DEV\SamsungDisplay\TDD-Ex1>
```

The status bar at the bottom shows the file path `TDD-Ex1 > calculator.py`, the line number `4:1`, the encoding `CRLF`, the file size `UTF-8`, the indentation `4 spaces`, the Python version `Python 3.10 virtualenv at C:\DEV\envs\py3_10_basic`, and a link to the Python documentation.

10. TDD 사이클 (Red-Green-Refactor)

TDD(Test-Driven Development)와 Refactoring을 통해 Clean Code를 달성하는 과정

예제: 숫자 더하기 계산기

3. 정제(Refine, Refactoring): 코드 개선

- 동작은 그대로 두고, 코드를 더 명확하고 깔끔하게 다듬습니다.

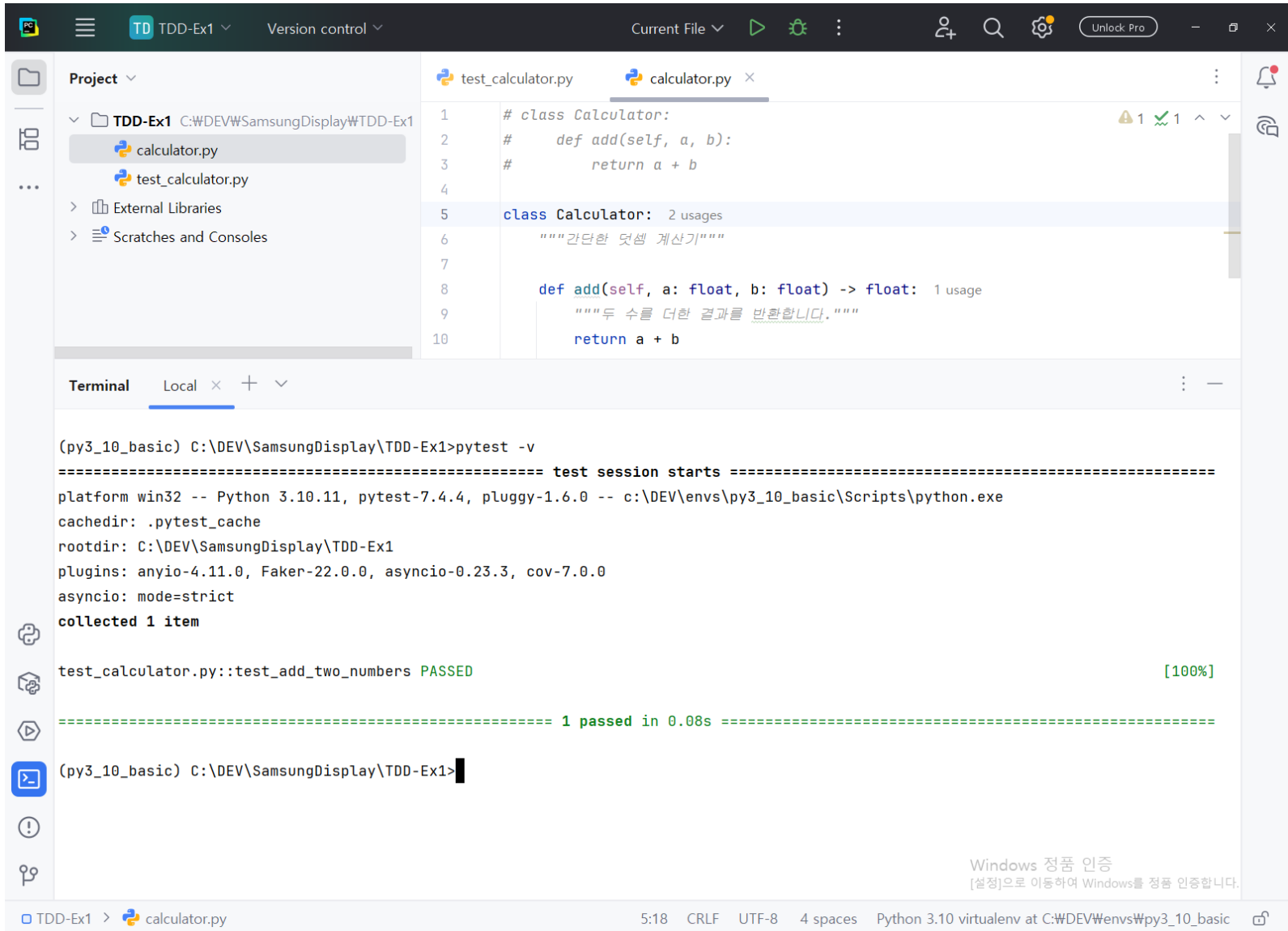
calculator.py

```
class Calculator:
    """간단한 덧셈 계산기"""

    def add(self, a: float, b: float) -> float:
        """두 수를 더한 결과를 반환합니다."""
        return a + b
```

테스트 다시 실행 → 그대로 통과
리팩토링 후에도 테스트가 깨지지 않는다는 건, 안전하게 코드를 개선했다는 의미입니다.

10. TDD 사이클 (Red-Green-Refactor)



The screenshot displays a code editor with two files: `test_calculator.py` and `calculator.py`. The `calculator.py` file contains a `Calculator` class with an `add` method. The `test_calculator.py` file contains a test for the `add` method. The terminal shows the execution of `pytest -v`, which passes the test.

```
# class Calculator:
#     def add(self, a, b):
#         return a + b

class Calculator: 2 usages
    """간단한 덧셈 계산기"""

    def add(self, a: float, b: float) -> float: 1 usage
        """두 수를 더한 결과를 반환합니다."""
        return a + b
```

```
(py3_10_basic) C:\DEV\SamsungDisplay\TDD-Ex1>pytest -v
===== test session starts =====
platform win32 -- Python 3.10.11, pytest-7.4.4, pluggy-1.6.0 -- c:\DEV\envs\py3_10_basic\Scripts\python.exe
cachedir: .pytest_cache
rootdir: C:\DEV\SamsungDisplay\TDD-Ex1
plugins: anyio-4.11.0, Faker-22.0.0, asyncio-0.23.3, cov-7.0.0
asyncio: mode=strict
collected 1 item

test_calculator.py::test_add_two_numbers PASSED [100%]

===== 1 passed in 0.08s =====

(py3_10_basic) C:\DEV\SamsungDisplay\TDD-Ex1>
```

Windows 정품 인증
[설정]으로 이동하여 Windows를 정품 인증합니다.

10. TDD 사이클 (Red-Green-Refactor)

TDD(Test-Driven Development)와 Refactoring을 통해 Clean Code를 달성하는 과정

예제: 숫자 더하기 계산기

4. 반복(Repeat): 새로운 요구사항 추가
- 새로운 요구사항을 반영한 테스트를 작성하여 TDD 사이클을 반복합니다.

추가된 테스트

```
def test_add_multiple_numbers():  
    calc = Calculator()  
    result = calc.add_multiple([1, 2, 3, 4])  
    assert result == 10
```

테스트 실패

- 기존 구현에서는 세미콜론(;)을 처리할 수 없기 때문에 테스트가 실패합니다.

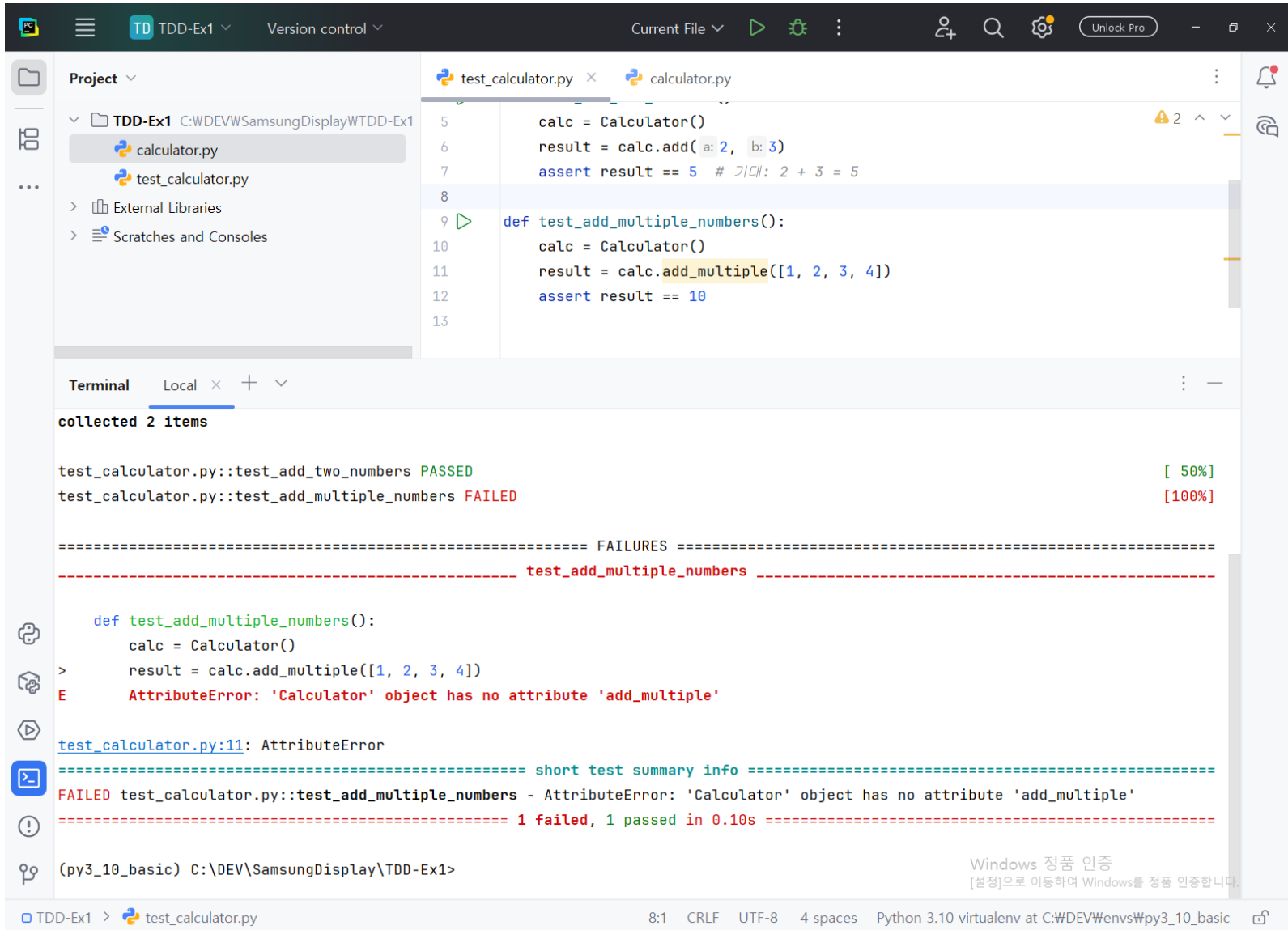
실행 결과

E AttributeError: 'Calculator' object has no attribute 'add_multiple'

다시 RED ●

이제 새 기능을 구현해봅시다.

10. TDD 사이클 (Red-Green-Refactor)



Project

- TDD-Ex1 C:\DEV\SamsungDisplay\TDD-Ex1
 - calculator.py
 - test_calculator.py
- External Libraries
- Scratches and Consoles

test_calculator.py

```
5 calc = Calculator()
6 result = calc.add(a: 2, b: 3)
7 assert result == 5 # 기대: 2 + 3 = 5
8
9 def test_add_multiple_numbers():
10     calc = Calculator()
11     result = calc.add_multiple([1, 2, 3, 4])
12     assert result == 10
13
```

calculator.py

```
5 calc = Calculator()
6 result = calc.add(a: 2, b: 3)
7 assert result == 5 # 기대: 2 + 3 = 5
8
9 def test_add_multiple_numbers():
10     calc = Calculator()
11     result = calc.add_multiple([1, 2, 3, 4])
12     assert result == 10
13
```

Terminal

collected 2 items

```
test_calculator.py::test_add_two_numbers PASSED [ 50%]
test_calculator.py::test_add_multiple_numbers FAILED [100%]
```

===== FAILURES =====

test_add_multiple_numbers

```
def test_add_multiple_numbers():
    calc = Calculator()
    result = calc.add_multiple([1, 2, 3, 4])
E   AttributeError: 'Calculator' object has no attribute 'add_multiple'
```

test_calculator.py:11: AttributeError

===== short test summary info =====

```
FAILED test_calculator.py::test_add_multiple_numbers - AttributeError: 'Calculator' object has no attribute 'add_multiple'
```

===== 1 failed, 1 passed in 0.10s =====

(py3_10_basic) C:\DEV\SamsungDisplay\TDD-Ex1>

Windows 정품 인증
[설정]으로 이동하여 Windows를 정품 인증합니다.

TDD-Ex1 > test_calculator.py

8:1 CRLF UTF-8 4 spaces Python 3.10 virtualenv at C:\DEV\envs\py3_10_basic

10. TDD 사이클 (Red-Green-Refactor)

TDD(Test-Driven Development)와 Refactoring을 통해 Clean Code를 달성하는 과정

예제: 숫자 더하기 계산기

4. 반복(Repeat): 다시 GREEN — 새 코드 추가

```
class Calculator:
    """간단한 덧셈 계산기"""

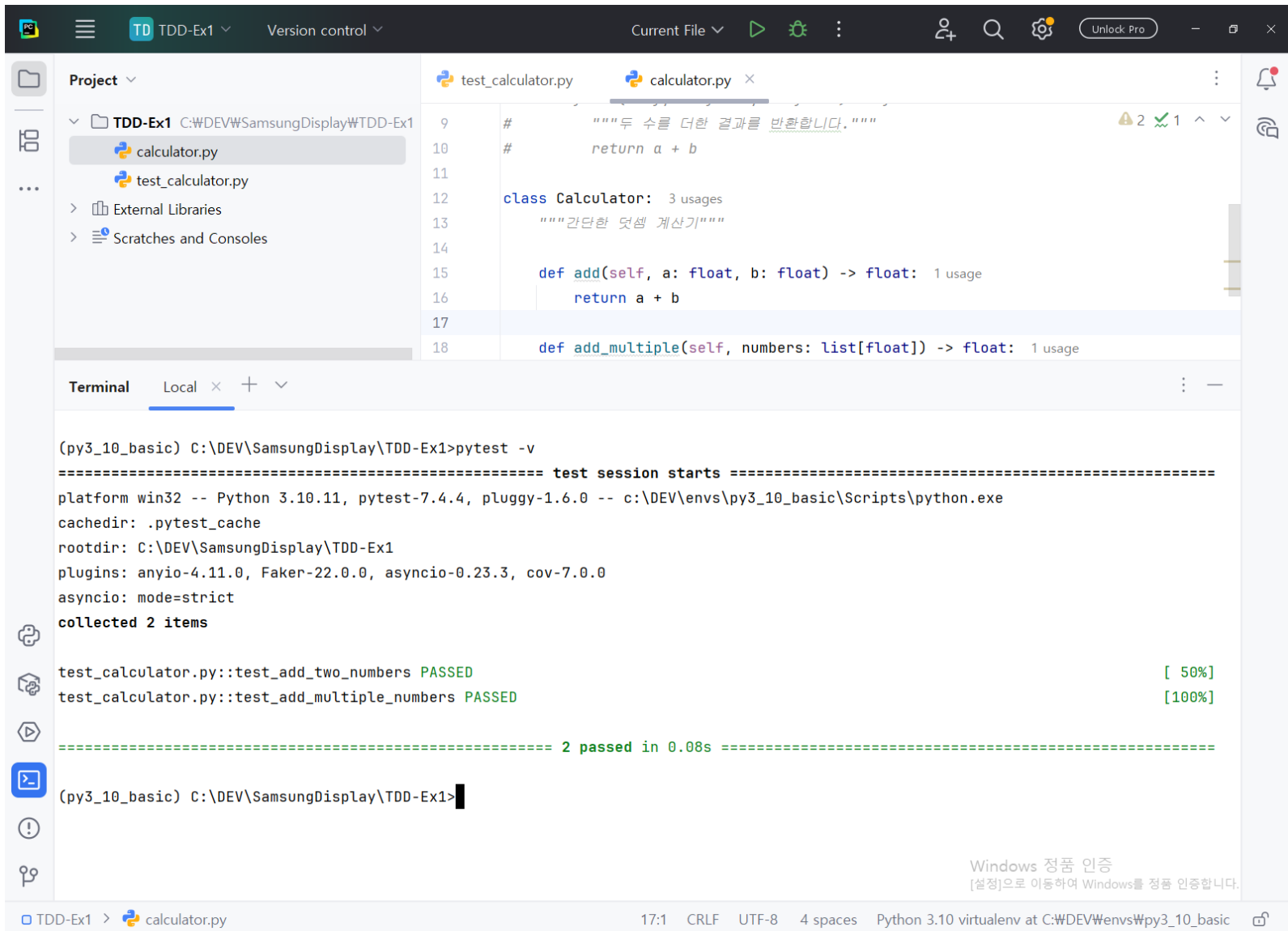
    def add(self, a: float, b: float) -> float:
        return a + b

    def add_multiple(self, numbers: list[float]) -> float:
        return sum(numbers)
```

테스트 통과:

```
test_calculator.py::test_add_multiple_numbers
PASSED
```

10. TDD 사이클 (Red-Green-Refactor)



The screenshot displays a code editor interface for a TDD (Test-Driven Development) cycle. The editor is titled 'TDD-Ex1' and shows a project structure with two files: 'calculator.py' and 'test_calculator.py'. The main editor window displays the 'calculator.py' file, which contains a class 'Calculator' with two methods: 'add' and 'add_multiple'. The 'add' method takes two float arguments and returns their sum. The 'add_multiple' method takes a list of floats and returns their sum. The terminal window at the bottom shows the output of a pytest command, indicating that two tests passed successfully.

```
test_calculator.py calculator.py x
9 # """두 수를 더한 결과를 반환합니다."""
10 # return a + b
11
12 class Calculator: 3 usages
13     """간단한 덧셈 계산기"""
14
15     def add(self, a: float, b: float) -> float: 1 usage
16         return a + b
17
18     def add_multiple(self, numbers: list[float]) -> float: 1 usage

Terminal Local x + v

(py3_10_basic) C:\DEV\SamsungDisplay\TDD-Ex1>pytest -v
===== test session starts =====
platform win32 -- Python 3.10.11, pytest-7.4.4, pluggy-1.6.0 -- c:\DEV\envs\py3_10_basic\Scripts\python.exe
cachedir: .pytest_cache
rootdir: C:\DEV\SamsungDisplay\TDD-Ex1
plugins: anyio-4.11.0, Faker-22.0.0, asyncio-0.23.3, cov-7.0.0
asyncio: mode=strict
collected 2 items

test_calculator.py::test_add_two_numbers PASSED [ 50%]
test_calculator.py::test_add_multiple_numbers PASSED [100%]

===== 2 passed in 0.08s =====

(py3_10_basic) C:\DEV\SamsungDisplay\TDD-Ex1>
```

Windows 정품 인증
[설정]으로 이동하여 Windows를 정품 인증합니다.

TDD-Ex1 > calculator.py 17:1 CRLF UTF-8 4 spaces Python 3.10 virtualenv at C:\DEV\envs\py3_10_basic



10. TDD 사이클 (Red-Green-Refactor)

TDD(Test-Driven Development)와 Refactoring을 통해 Clean Code를 달성하는 과정

예제: 숫자 더하기 계산기

5. 최종 코드

- TDD와 Refactoring을 통한 Clean Code 달성

1. 질문 (Ask):

- 요구사항을 반영한 테스트를 작성하여 시스템에 질문합니다.

2. 응답 (Respond):

- 테스트를 통과할 최소한의 코드를 작성하여 질문에 답합니다.

3. 정제 (Refine, Refactoring):

- 테스트가 성공한 코드를 리팩토링하여 가독성, 재사용성, 유지보수성을 개선합니다.

4. 반복 (Repeat):

- 새로운 요구사항이 추가될 때마다 TDD 사이클을 반복하여 시스템을 점진적으로 개선합니다.

```
from typing import Iterable
```

```
class Calculator:
```

```
    """클린 코드 스타일의 덧셈 계산기"""
```

```
    def add(self, a: float, b: float) -> float:
```

```
        """두 수를 더합니다."""
```

```
        return a + b
```

```
    def add_all(self, numbers: Iterable[float]) -> float:
```

```
        """여러 수를 모두 더합니다."""
```

```
        return sum(numbers)
```

10. TDD 사이클 (Red-Green-Refactor)

TDD(Test-Driven Development)와 Refactoring을 통해 Clean Code를 달성하는 과정

예제: 숫자 더하기 계산기

calculator.py

```
from typing import Iterable, Union
```

```
Number = Union[int, float]
```

```
class Calculator:
```

```
    """테스트 주도 개발(TDD)로 만든 덧셈 계산기"""
```

```
    def add(self, a: Number, b: Number) -> Number:
```

```
        """두 수를 더합니다."""
```

```
        return a + b
```

```
    def add_all(self, numbers: Iterable[Number]) -> Number:
```

```
        """리스트나 튜플의 모든 수를 더합니다."""
```

```
        return sum(numbers)
```

test_calculator.py

```
from calculator import Calculator
```

```
def test_add_two_numbers():
```

```
    calc = Calculator()
```

```
    assert calc.add(2, 3) == 5
```

```
def test_add_multiple_numbers():
```

```
    calc = Calculator()
```

```
    assert calc.add_all([1, 2, 3, 4]) == 10
```

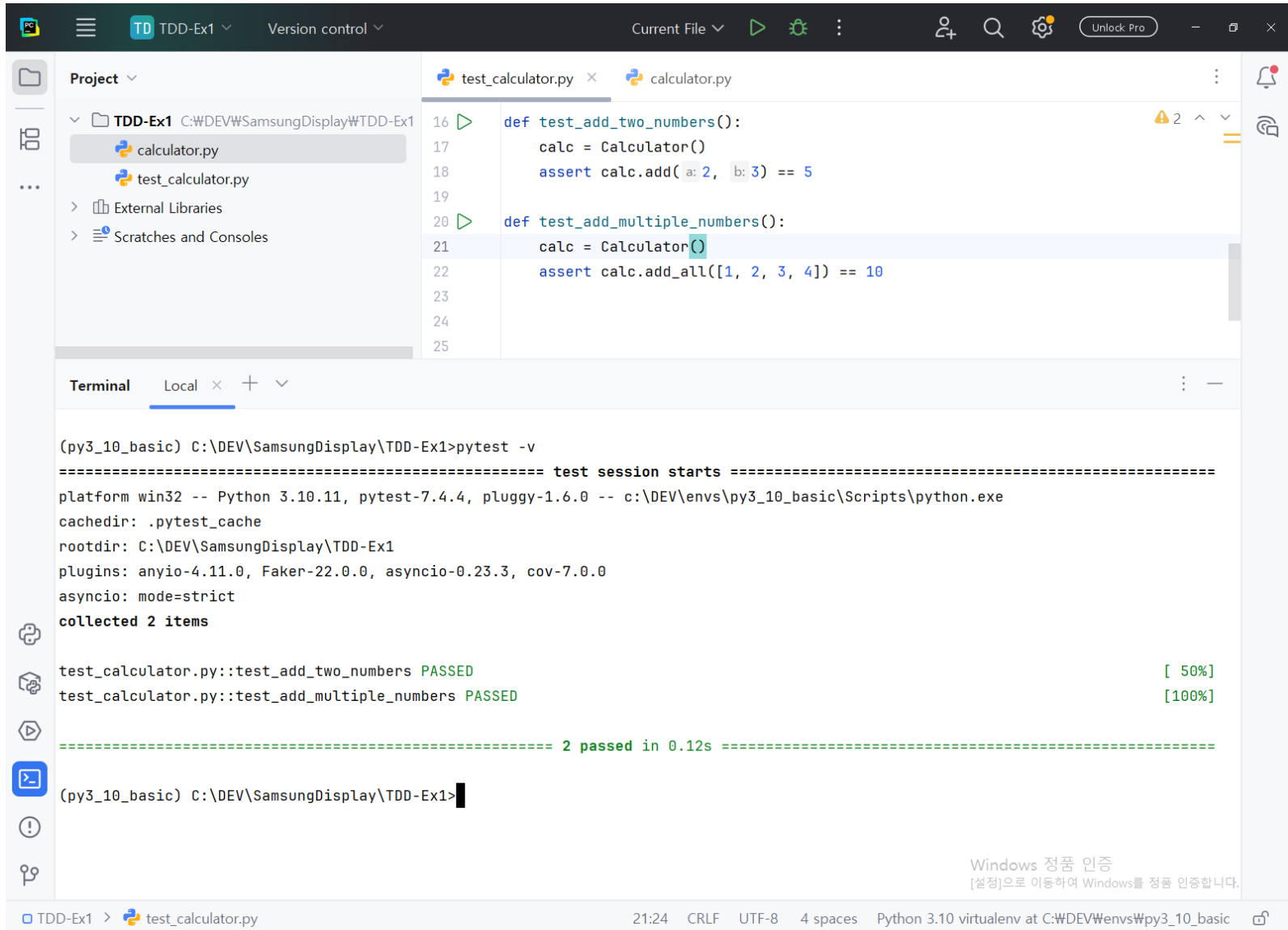
실행 결과

collected 2 items

test_calculator.py::test_add_two_numbers PASSED [50%]

test_calculator.py::test_add_multiple_numbers PASSED [100%]

10. TDD 사이클 (Red-Green-Refactor)



The screenshot displays a code editor interface with a project named 'TDD-Ex1'. The project structure shows two files: 'calculator.py' and 'test_calculator.py'. The 'test_calculator.py' file is open, showing two test functions: 'test_add_two_numbers()' and 'test_add_multiple_numbers()'. The 'test_add_multiple_numbers()' function is currently selected and highlighted. The terminal window at the bottom shows the execution of 'pytest -v', which successfully ran both tests, indicating a 'Green' state in the TDD cycle.

```
def test_add_two_numbers():
    calc = Calculator()
    assert calc.add(a: 2, b: 3) == 5

def test_add_multiple_numbers():
    calc = Calculator()
    assert calc.add_all([1, 2, 3, 4]) == 10
```

```
(py3_10_basic) C:\DEV\SamsungDisplay\TDD-Ex1>pytest -v
===== test session starts =====
platform win32 -- Python 3.10.11, pytest-7.4.4, pluggy-1.6.0 -- c:\DEV\envs\py3_10_basic\Scripts\python.exe
cachedir: .pytest_cache
rootdir: C:\DEV\SamsungDisplay\TDD-Ex1
plugins: anyio-4.11.0, Faker-22.0.0, asyncio-0.23.3, cov-7.0.0
asyncio: mode=strict
collected 2 items

test_calculator.py::test_add_two_numbers PASSED [ 50%]
test_calculator.py::test_add_multiple_numbers PASSED [100%]

===== 2 passed in 0.12s =====

(py3_10_basic) C:\DEV\SamsungDisplay\TDD-Ex1>
```

Windows 정품 인증
[설정]으로 이동하여 Windows를 정품 인증합니다.

TDD-Ex1 > test_calculator.py 21:24 CRLF UTF-8 4 spaces Python 3.10 virtualenv at C:\DEV\envs\py3_10_basic



11. 실습

| 실습2~ 실습 5

실습2 : 파이썬 테스트 코드 작성해보기

실습3 : 파이썬 테스트 코드 나눠서 테스트 하기

실습4 : TDD-문자열 계산기

『2과목』

클린코드 + 테스트 기반 구현

- Part 1. 오리엔테이션 및 목표
- Part 2. 클린코드 핵심 원칙
- Part 3. 테스트 기반 개발
- Part 4. 종합 실습 & 마무리





1. Full-Stack 사전 준비

| OrderBean Toy-Project 개요

프로젝트: OrderBean - 커피 주문 관리 시스템

작성일: 2025년 11월 22일

목적: Cursor AI를 활용한 TDD(Test-Driven Development)
개발 프랙티스

1. Full-Stack 사전 준비

| TDD란?

TDD(Test-Driven Development)는 테스트를 먼저 작성하고, 그 테스트를 통과하는 최소한의 코드를 구현한 뒤, 리팩토링하는 개발 방법론입니다.

 RED →  GREEN →  REFACTOR →  REPEAT



1. Full-Stack 사전 준비

TDD 7단계 프로세스

1. Planning (요구사항 → 테스트 단위 변환)



2. RED (실패하는 테스트 작성)



3. AI Assist (테스트 확장 및 보완)



4. GREEN (최소 구현으로 테스트 통과)



5. Review (코드 리뷰 및 개선 포인트 도출)



6. Refactor (테스트 기반 리팩토링)



7. Loop (기능 추가 시 2번으로 반복)

1. Full-Stack 사전 준비

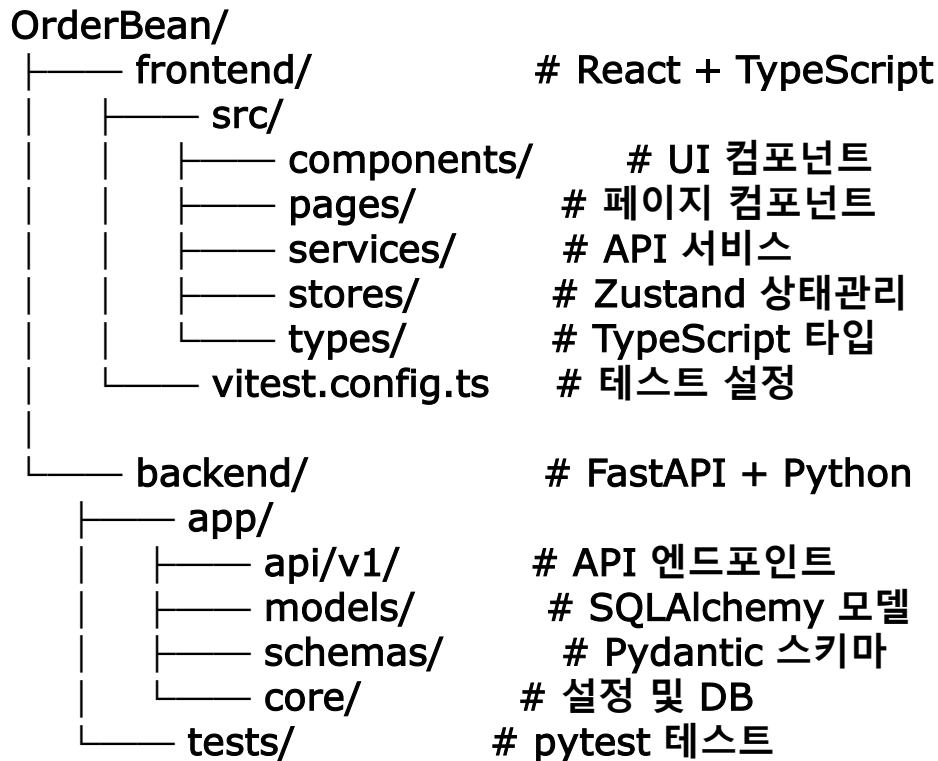
TDD 7단계 프로세스 단계별 요약

단계	목적	산출물	Cursor AI 역할
1. Planning	요구사항 분석	테스트 케이스 목록	요구사항 분석, 테스트 케이스 도출
2. RED	실패 테스트 작성	실패하는 테스트 코드	테스트 코드 초안 생성
3. AI Assist	테스트 확장	완성된 테스트 코드	엡지 케이스, 에러 케이스 추가
4. GREEN	최소 구현	테스트 통과하는 코드	구현 코드 생성
5. Review	품질 검토	개선 포인트 목록	코드 리뷰, 제안
6. Refactor	코드 개선	리팩토링된 코드	리팩토링 제안
7. Loop	반복	다음 기능	다음 테스트 케이스 제안



1. Full-Stack 사전 준비

OrderBean 예상 프로젝트 구조





1. Full-Stack 사전 준비

기술 스택

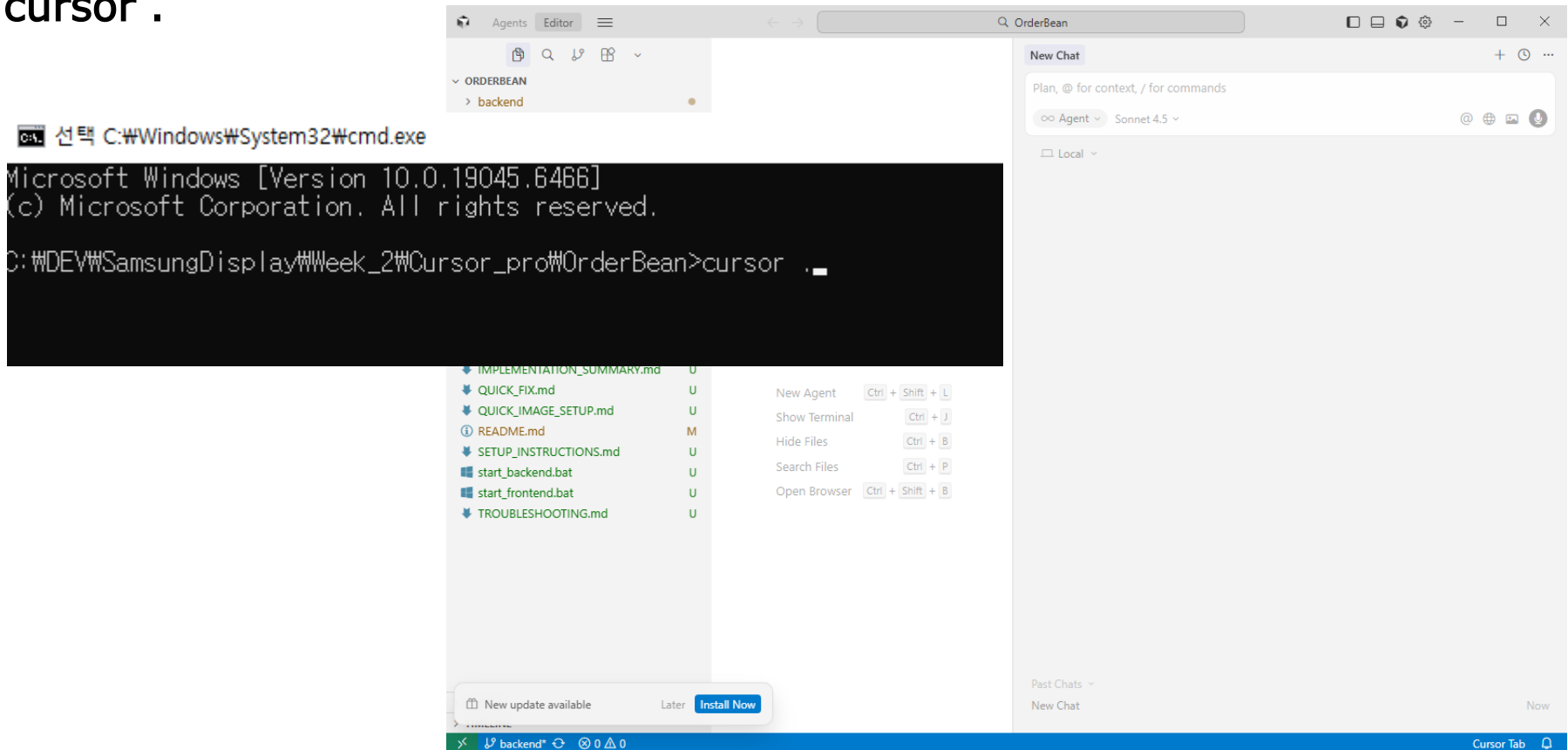
영역	기술	테스트 도구
Frontend	React 18 + TypeScript	Vitest + React Testing Library
Backend	FastAPI + Python 3.11	pytest + pytest-asyncio
Database	PostgreSQL 15	pytest fixtures
State	Zustand	Vitest

1. Full-Stack 사전 준비

Cursor AI 설정

Cursor AI에서 프로젝트 열기

- # OrderBean 프로젝트 폴더에서 cursor .





1. Full-Stack 사전 준비

| Cursor AI 설정

권장 Cursor 설정:

- Cmd/Ctrl + I: AI Chat 열기
- Cmd/Ctrl + K: 인라인 코드 생성
- Cmd/Ctrl + L: 선택한 코드에 AI 적용



1. Full-Stack 사전 준비

테스트 환경 확인

Frontend 테스트 환경:

```
cd frontend
```

```
npm test # Vitest watch 모드
```

```
npm test -- --run # 단일 실행
```

Backend 테스트 환경:

```
cd backend
```

```
pytest # 전체 테스트
```

```
pytest -v # 상세 출력
```

```
pytest -k "test_menu" # 특정 테스트만
```

1. Full-Stack 사전 준비

| 참고 문서 준비

Cursor AI에 컨텍스트로 제공할 문서들:

- project-requirements.md - 요구사항 명세
- Frontend_UI_PRD_Customer.md - 고객 화면 PRD
- Frontend_UI_PRD_Admin.md - 관리자 화면 PRD
- OrderBean_Mermaid_Diagrams.md - 시스템 다이어그램

2. Full-Stack Planning + Cursor AI 개발

Cursor AI 프롬프트 - Planning 단계

1단계: Planning (요구사항 → 테스트 단위 변환)

컨텍스트

- 프로젝트: OrderBean 커피 주문 시스템
- 기술 스택: [Frontend: React + TypeScript + Vitest / Backend: FastAPI + pytest]
- 현재 작업: [구현하려는 기능명]

요청

다음 요구사항을 테스트 케이스로 변환해주세요:

요구사항

[PRD 또는 요구사항 명세에서 해당 기능 복사]

원하는 출력

1. 기능을 테스트 가능한 단위로 분해
2. 각 테스트 케이스의 describe/it 구조
3. 테스트 우선순위 (P0: 필수, P1: 중요, P2: 선택)

예시 형식

```
```typescript
describe('[컴포넌트명]', () => {
 describe('[기능 그룹]', () => {
 it('[P0] 테스트 설명 1');
 it('[P1] 테스트 설명 2');
 });
});
```



## 2. Full-Stack Planning + Cursor AI 개발

### 실전 예시

#### ## 컨텍스트

- 프로젝트: OrderBean 커피 주문 시스템
- 기술 스택: Frontend: React + TypeScript + Vitest
- 현재 작업: MenuCard 컴포넌트 구현

#### ## 요구사항

1. 메뉴 이름, 가격, 설명 표시
2. 옵션 체크박스 (샷 추가, 시럽 추가)
3. 옵션 선택 시 가격 동적 계산
4. "담기" 버튼으로 장바구니 추가
5. 이미지 표시 (없으면 플레이스홀더)

이 요구사항을 테스트 케이스로 변환해주세요.



## 2. Full-Stack Planning + Cursor AI 개발

### 실전 예시 좀 더 구체화 ...

## 요구사항

MenuCard 컴포넌트 구현:

- 메뉴 이름, 가격, 설명 표시
- 옵션 체크박스 (샷 추가 +500원, 시럽 추가 +0원)
- 옵션 선택 시 가격 동적 계산
- "담기" 버튼으로 장바구니 추가
- 이미지 표시 (없으면 플레이스홀더)

이 요구사항을 테스트 케이스로 변환해주세요.



# 3. Full-Stack TDD + RED + AI Assist Cursor AI .

## 실패하는 테스트 작성

### # Cursor AI 프롬프트 - RED 단계

## 컨텍스트

- 파일 위치: [테스트 파일 경로]
- 테스트 프레임워크: [Vitest / pytest]

## 요청

다음 테스트 케이스에 대한 **\*\*실패하는 테스트 코드\*\***를 작성해주세요.

### 테스트 케이스

[1단계에서 도출한 테스트 케이스 목록]

### 요구사항

1. 아직 구현되지 않은 컴포넌트/함수를 import
2. 테스트는 실행되지만 FAIL 해야 함
3. 명확한 테스트 설명 작성
4. AAA 패턴 (Arrange-Act-Assert) 사용

### 사용할 테스트 도구

- React Testing Library (screen, render, userEvent)
- Vitest (describe, it, expect, vi)



# 3. Full-Stack TDD + RED + AI Assist Cursor AI .

## 실전 예시

### ## 컨텍스트

- 파일 위치: frontend/src/components/customer/MenuCard.test.tsx
- 테스트 프레임워크: Vitest + React Testing Library

### ## 테스트 케이스

1. 메뉴 이름이 표시되어야 한다
2. 메뉴 가격이 표시되어야 한다
3. 옵션 체크박스가 표시되어야 한다
4. 옵션 선택 시 가격이 업데이트되어야 한다
5. 담기 버튼 클릭 시 onAddToCart가 호출되어야 한다

실패하는 테스트 코드를 작성해주세요.



## 3. Full-Stack TDD + RED + AI Assist Cursor AI .

### | 실전 예시 좀 더 구체화 ...

위 테스트 케이스에 대한 실패하는 테스트 코드를 작성해주세요.

사용 도구: Vitest + React Testing Library





# 3. Full-Stack TDD + RED + AI Assist Cursor AI .

## 테스트 확장 및 보완

### # Cursor AI 프롬프트 - AI Assist 단계

## 컨텍스트

현재 작성된 테스트 코드:

```
```typescript
```

[현재 테스트 코드 붙여넣기]

```
```
```

## 요청

테스트 코드를 검토하고 다음을 추가해주세요:

### 확장 요청

1. **\*\*엣지 케이스\*\***: 경계값, null/undefined 처리
2. **\*\*에러 케이스\*\***: 예외 상황, 네트워크 오류
3. **\*\*접근성 테스트\*\***: ARIA 속성, 키보드 네비게이션
4. **\*\*성능 관련\*\***: 대량 데이터, 빠른 연속 클릭

### 체크리스트

- [ ] 빈 데이터 처리
- [ ] 최대/최소 값 처리
- [ ] 비동기 작업 테스트
- [ ] 에러 바운더리 테스트
- [ ] 로딩 상태 테스트



# 3. Full-Stack TDD + RED + AI Assist Cursor AI

## 실전 예시

## 현재 테스트 코드

[MenuCard.test.tsx의 기본 테스트 코드]

## 확장 요청

MenuCard 컴포넌트에 대해 다음 케이스를 추가해주세요:

1. 옵션이 없는 메뉴 처리
2. 재고가 0인 메뉴 (담기 버튼 비활성화)
3. 이미지 로드 실패 시 플레이스홀더 표시
4. 가격이 0원인 경우 처리
5. 메뉴명이 매우 긴 경우 (텍스트 오버플로우)



## 3. Full-Stack TDD + RED + AI Assist Cursor AI .

### 실전 예시 좀 더 구체화 ...

위 테스트에 다음 엡지 케이스를 추가해주세요:

1. 옵션이 없는 메뉴
2. 재고가 0인 메뉴
3. 이미지 로드 실패



## 2. Day 3 Preview

### DAY 3 예고

**Full-Stack GREEN + Review Cursor AI 개발**

# THANK YOU.

앞으로의 엔지니어는 단순한 '코더'나 '기계 조작자'가 아니라 뇌-기계 인터페이스를 통해 지식과 능력을 즉각 확장하는 존재(뉴로-인터페이스: Neuro Interface)가 될 수 있습니다.

- 🎯 목표 달성을 위한 여정이 시작됩니다.
- 🌟 궁금한 점이 있으시면 언제든지 문의해주세요!
- 🚀 함께 코더와 프롬프트 전문가로 성장해 나갑시다!